REȚELE DE CALCULATOARE · SEMESTRUL 1 · CURSUL 5

Nivelul aplicație III: poșta electronică, Usenet, DNS și NAT

Pachet didactic pentru săptămâna 5: note de curs, seminar, laborator Linux și laborator Windows (câte două ore fiecare)

Conf. univ. dr. Nicolaie Popescu-Bodorin, 2026
+40722882471, bodorin(on)live.com, https://lmrec.org/bodorin/

Partea	Conținut	Durată
A	Cursul 5 — Nivelul aplicație III: SMTP, formatul mesajelor și MIME, POP3 și IMAP, Usenet, DNS și NAT	2 h
B	Seminarul 5 — Mesaje, nume și adrese translatate	2 h
C	Laboratorul 5 (Linux) — Poștă de mână, DNS octet cu octet peste UDP, un router NAT propriu	2 h
D	Laboratorul 5 (Windows) — Aceleași protocoale cu instrumente Windows și Winsock; găsirea propriului NAT	2 h

Partea A · Cursul 5

După acest curs ar trebui să puteți

- numi componentele sistemului de poștă electronică și urmări un mesaj de la expeditor la cititor;
- citi și scrie un dialog SMTP și distinge plicul (envelope) de antetul mesajului;
- explica formatul unui mesaj de poștă și modul în care MIME transportă atașamente și text non-ASCII;
- compara POP3 și IMAP și spune cum este protejată poșta împotriva interceptării și a falsificării;
- descrie Usenet și modul în care serverele NNTP își propagă articolele prin inundare (flooding);
- explica DNS: serviciile, ierarhia, rezolvarea iterativă și recursivă, cache-ul, înregistrările de resurse și formatul mesajelor;
- explica modul de funcționare a NAT, de ce există și ce aplicații afectează.

Planul celor două ore

Partea	Subiect	Timp
Ora 1		
1	Ultima săptămână a nivelului aplicație	3 min
2	Poșta electronică: sistemul	7 min
3	SMTP: împingerea poștei	15 min
4	Mesajul: formatul și MIME	10 min

Partea	Subiect	Timp
5	Citirea poștei: POP3, IMAP și webmail; protejarea poștei	15 min
	Pauză	10 min
Ora 2		
6	Usenet: știri prin inundare	8 min
7	DNS: servicii și ierarhie	12 min
8	DNS: rezolvare, cache, înregistrări și mesaje	15 min
9	NAT: multe host-uri în spatele unei singure adrese; rezumat	15 min

1. Ultima săptămână a nivelului aplicație

Săptămânile 3 și 4 au acoperit HTTP, HTTPS, TELNET, SSH și FTP. Săptămâna aceasta completează lista din programa analitică: protocoalele de poștă electronică **SMTP**, **POP** și **IMAP**, sistemul de știri **Usenet**, **sistemul de nume de domenii (DNS)**, de care depinde aproape orice alt protocol, și **translatarea adreselor de rețea (NAT)**, care nu este deloc un protocol de aplicație, ci o funcție a nivelului rețea cu care trebuie să conviețuiască orice protocol de aplicație. Strict vorbind, NAT ține de nivelul rețea — îl întâlnim acum din cauza efectelor sale asupra aplicațiilor și vom reveni acolo asupra mecanismelor sale.

2. Poșta electronică: sistemul

Poșta electronică (1971) este mai veche decât TCP/IP și rămâne una dintre cele mai folosite aplicații Internet. Spre deosebire de Web, este **asincronă**: cititorul nu trebuie să fie conectat atunci când mesajul este trimis. Componentele sale principale sunt:

- **Agenții utilizator** (programele de poștă) — Thunderbird, Outlook, aplicația Mail a unui telefon sau o pagină de webmail — cu care oamenii scriu și citesc mesaje.
- **Serverele de poștă**, fiecare deținând **cutiile poștale** ale utilizatorilor săi și o **coadă** de mesaje de ieșire. Un server care acceptă mesajul nou al unui utilizator este un *server de trimitere (submission)*; un server care retransmite poșta între domenii este un *agent de transfer al poștei (MTA)*.
- **Protocoalele**: **SMTP** transportă un mesaj de la agentul utilizator al expeditorului la serverul acestuia și de la server la server — el *împinge (push)*. **POP3** și **IMAP** îi permit agentului utilizator al cititorului să preia mesajul din cutia poștală — ele *trag (pull)*. **DNS** îi spune unui server expeditor ce server primește poșta pentru un domeniu.

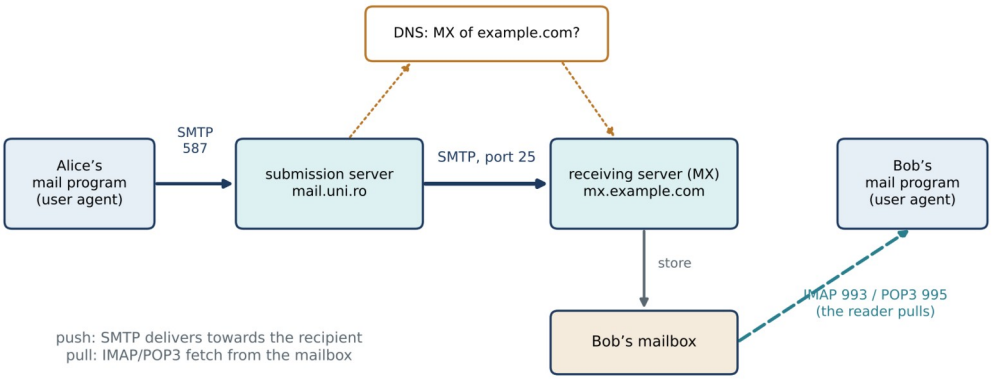


Figura 1. Traseul unui mesaj. SMTP îl împinge de la agentul utilizator al lui Alice la serverul furnizorului ei și de acolo la serverul indicat în înregistrarea MX a domeniului destinatarului; agentul utilizator al lui Bob îl trage mai târziu cu IMAP sau POP3.

De ce nu se trimite direct de pe laptopul lui Alice pe cel al lui Bob? Pentru că cutia poștală a lui Bob trebuie să fie accesibilă atunci când Bob nu este, iar laptopul lui Alice trebuie să poată preda mesajul și să se deconecteze: dacă serverul destinatar este căzut, serverul lui Alice păstrează mesajul în coadă și reîncearcă, de obicei la fiecare jumătate de oră timp de mai multe zile, înainte de a-i trimite lui Alice un raport de eroare.

3. SMTP: împingerea poștei

SMTP (Simple Mail Transfer Protocol; RFC 821, 1982; versiunea actuală RFC 5321) rulează peste TCP. Ca și HTTP, este un protocol text de comenzi și răspunsuri cu trei cifre; spre deosebire de HTTP, clientul împinge mesajul către server, iar mai multe mesaje se pot succeda pe aceeași conexiune. Un dialog complet, așa cum îl veți tasta în laborator:

```
S: 220 mx.example.com ESMTP ready
C: EHLO mail.uni.ro
S: 250-mx.example.com
S: 250-SIZE 52428800
S: 250 STARTTLS
C: MAIL FROM:<alice@uni.ro>
S: 250 OK
C: RCPT TO:<bob@example.com>
S: 250 OK
C: DATA
S: 354 End data with <CR><LF>.<CR><LF>
C: From: Alice <alice@uni.ro>
C: To: Bob <bob@example.com>
C: Subject: Lab 5
C:
C: Hello Bob, see you on Monday.
C: .
S: 250 OK: queued as 7F3A2
C: QUIT
S: 221 Bye
```

Comandă	Semnificație
`HELO / EHLO nume`	salut; EHLO (SMTP extins) îi cere serverului să-și enumere **extensiile**: SIZE, 8BITMIME, STARTTLS, AUTH, SMTPUTF8, …
`MAIL FROM:<adresă>`	începe o tranzacție; **calea inversă (reverse path)**, către care merg rapoartele de eroare (bounces)

Comandă	Semnificație
`RCPT TO:<adresă>`	un destinatar; se repetă pentru fiecare destinatar de pe acest server
`DATA`	urmează mesajul, încheiat printr-o linie care conține un singur punct
`RSET`, `NOOP`, `QUIT`	abandonează tranzacția; nu face nimic; încheie sesiunea
`STARTTLS`, `AUTH`	comută conexiunea pe TLS (RFC 3207); autentifică utilizatorul (RFC 4954)

Răspunsurile urmează schema FTP: 2xx succes (220 pregătit, 250 OK, 221 închidere), 3xx trimiteți mai mult (354 începeți introducerea mesajului), 4xx eșec temporar — încercați mai târziu (421, 450 cutie poștală ocupată, 451), 5xx eșec permanent (550 utilizator inexistent, 552 prea mare, 554 respins).

3.1 Trei detalii care contează

- **Încadrarea.** Mesajul se termină la o linie care conține doar un punct. O linie a mesajului care începe cu un punct este trimisă cu un punct suplimentar (**dot-stuffing**), pe care receptorul îl elimină — metoda delimitatorului din Cursul 3, cu caracter de evadare (escaping).
- **Plicul și antetul sunt diferite.** `MAIL FROM` și `RCPT TO` formează **plicul (envelope)**, folosit pentru livrare, ca adresa de pe un plic de hârtie. Liniile `From:` și `To:` din interiorul mesajului reprezintă **antetul (header)**, afișat cititorului, ca antetul unei scrisori. Ele pot diferi: un destinatar Bcc apare în plic, dar în nicio linie de antet; o listă de discuții livrează către mii de adrese de plic un mesaj al cărui `To:` numește lista. Iar nimic din SMTP-ul simplu nu le verifică pe niciunele — motiv pentru care există expeditori falsificați (secțiunea 5.3).
- **Porturi. 25** pentru retransmiterea de la server la server; **587** pentru trimiterea de către agenții utilizator, după autentificare (RFC 6409); **465** pentru trimitere prin TLS încă de la primul octet (RFC 8314). Majoritatea ISP-urilor blochează portul 25 de ieșire pentru conexiunile casnice, astfel încât PC-urile infectate să nu poată trimite spam direct.

3.2 Găsirea serverului destinatar

Serverul lui Alice trebuie să afle ce server primește poșta pentru *example.com*. Întreabă DNS-ul de înregistrările **MX** (mail exchanger) ale domeniului, de exemplu `example.com. MX 10 mx1.example.com.` și `MX 20 mx2.example.com.`, încearcă mai întâi serverul cu cel mai mic număr de preferință și apoi, la nevoie, pe celelalte. Acesta este primul nostru exemplu de aplicație care nu poate funcționa fără DNS.

3.3 Comparație între SMTP și HTTP

	HTTP	SMTP
Direcție	pull: clientul cere un obiect	push: clientul livrează un mesaj
Conexiune	persistentă (HTTP/1.1)	persistentă: mai multe mesaje pe conexiune
Codificare	orice octeți în corp (Content-Length)	inițial linii ASCII pe 7 biți; datele binare necesită codificare MIME sau extensia 8BITMIME
Mai multe obiecte	câte un răspuns pentru fiecare obiect	toate părțile într-un singur mesaj multipart
Răspunsuri	coduri de stare de trei cifre	coduri de răspuns de trei cifre

4. Mesajul: formatul și MIME

SMTP doar transportă mesajul; formatul acestuia este definit separat în **RFC 5322**: linii de text terminate cu CR LF, un **antet** format din câmpuri `Nume: valoare`, o linie goală și un **corp**. Principalele câmpuri de antet sunt `From`, `To`, `Cc`, `Subject`, `Date`, `Message-ID` (un identificator unic la nivel mondial), `Reply-To`, `In-Reply-To` și `References` (care leagă mesajele unei conversații). Fiecare server care retransmite mesajul adaugă un câmp **Received** în partea de sus, astfel încât antetul înregistrează traseul mesajului, cu cel mai recent salt (hop) primul.

RFC 5322 permite doar text ASCII. **MIME** (Multipurpose Internet Mail Extensions, RFC-urile 2045–2049) adaugă tot restul prin câteva câmpuri de antet suplimentare:

Câmp	Semnificație
`MIME-Version: 1.0`	mesajul folosește MIME
`Content-Type`	tipul media: `text/plain; charset=utf-8`, `text/html`, `image/jpeg`, `application/pdf` sau `multipart/mixed; boundary="…"` pentru mai multe părți — aceleași tipuri media pe care le folosește HTTP
`Content-Transfer-Encoding`	modul în care octeții sunt scriși ca text: `7bit`, `8bit`, `quoted-printable` (pentru text în mare parte ASCII: =C8 pentru un octet 0xC8) sau **`base64`** (pentru date binare)
`Content-Disposition`	`inline` sau `attachment; filename="report.pdf"`

```
MIME-Version: 1.0
Content-Type: multipart/mixed; boundary="==B7f2"

--==B7f2
Content-Type: text/plain; charset="utf-8"

Hello Bob, the report is attached.
--==B7f2
Content-Type: application/pdf
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="report.pdf"

JVBERi0xLjcKJcfsj6IKNSAwIG9iago8PC9MZW5ndGggNiAwIFIvRmlsdGVy...
--==B7f2--
```

base64 scrie fiecare grup de 3 octeți ca 4 caractere dintr-un alfabet de 64 de litere (A–Z, a–z, 0–9, +, /), în linii de cel mult 76 de caractere. Un atașament crește așadar cu o treime, plus terminatorii de linie — aproximativ **37 %**: un fișier de 10 MB devine un mesaj de aproximativ 13,7 MB, unul dintre motivele pentru care serverele de poștă limitează dimensiunea mesajelor. Textul non-ASCII din câmpurile de antet, cum ar fi un subiect în limba română, este scris ca un **cuvânt codificat (encoded word)** (RFC 2047): `Subject: =?utf-8?b?TGFib3JhdG9ydWwgNQ==?=`.

5. Citirea poștei: POP3, IMAP și webmail

SMTP nu poate fi folosit pentru citirea poștei: el împinge, iar calculatorul cititorului este adesea deconectat sau în spatele NAT. De aceea agentul utilizator al cititorului *trage* mesajele din cutia sa poștală cu un protocol de acces.

5.1 POP3

POP3 (Post Office Protocol versiunea 3; RFC 1939; portul 110 sau 995 peste TLS) este simplu în mod deliberat. O sesiune trece prin trei stări:

- **AUTHORIZATION** (autorizare): `USER bob`, `PASS secret` (fiecare primește răspunsul `+OK` sau `-ERR`).
- **TRANSACTION** (tranzacție): `STAT` (numărul și dimensiunea totală a mesajelor), `LIST` (dimensiunea fiecăruia), `UIDL` (un identificator unic pentru fiecare mesaj), `RETR n` (mesajul, încheiat printr-o linie cu un singur punct, cu dot-stuffing ca în SMTP), `TOP n k` (antetul și k linii), `DELE n` (marcare pentru ștergere), `RSET` (anularea marcajelor).
- **UPDATE** (actualizare): după `QUIT`, serverul elimină mesajele marcate pentru ștergere.

POP3 se potrivește unui singur dispozitiv care descarcă poșta și o păstrează — *descarcă și șterge* sau *descarcă și păstrează*, cu UIDL pentru a evita descărcarea de două ori. Nu știe nimic despre foldere, marcaje de citire sau alte dispozitive.

5.2 IMAP

IMAP (Internet Message Access Protocol; IMAP4rev1 RFC 3501, IMAP4rev2 RFC 9051; portul 143 sau 993 peste TLS) păstrează poșta **pe server**, organizată în **foldere**, cu **marcaje (flags)** pentru fiecare mesaj (`\Seen`, `\Answered`, `\Flagged`, `\Deleted`). Fiecare dispozitiv vede aceeași stare, ceea ce așteaptă oamenii care au un telefon, un laptop și un cont de webmail. Clientul poate căuta pe server și poate prelua doar ce îi trebuie — antetele unui folder, o parte a unui mesaj — util pe o legătură mobilă lentă. Comenzile poartă o **etichetă (tag)** aleasă de client, astfel încât mai multe pot fi în curs simultan; serverul trimite linii de date neetichetate (`*`) și încheie fiecare comandă cu o linie care poartă eticheta acesteia:

```
C: a1 LOGIN bob secret
S: a1 OK LOGIN completed
C: a2 SELECT INBOX
S: * 3 EXISTS
S: * 1 RECENT
S: a2 OK [READ-WRITE] SELECT completed
C: a3 FETCH 3 (FLAGS BODY.PEEK[HEADER.FIELDS (FROM SUBJECT)])
S: * 3 FETCH (FLAGS () BODY[HEADER.FIELDS (FROM SUBJECT)] {52}
S: From: Alice <alice@uni.ro>
S: Subject: Lab 5
S: )
S: a3 OK FETCH completed
C: a4 STORE 3 +FLAGS (\Seen)
C: a5 LOGOUT
```

`{52}` anunță că urmează 52 de octeți de date literale — un prefix de lungime în interiorul unui protocol bazat pe linii. Cu comanda `IDLE` (RFC 2177), serverul îl poate anunța imediat pe client despre poșta nouă, în loc să fie interogat periodic (polling).

5.3 Webmail și protejarea poștei

Cu **webmail** (Gmail, Outlook.com, serviciul de webmail al unei universități), agentul utilizator este o pagină web: browserul vorbește HTTPS cu serverul de webmail, care folosește SMTP pentru trimitere și citește el însuși cutia poștală; aplicațiile mobile folosesc adesea protocoale proprietare sau **JMAP** (RFC-urile 8620–8621), un înlocuitor modern al IMAP, bazat pe JSON peste HTTPS.

Poșta are nevoie de două tipuri de protecție. Împotriva **interceptării**: agenții utilizator folosesc TLS pe porturile 587/465/993/995, iar serverele își actualizează conexiunile pe portul 25 cu STARTTLS — dar acest lucru protejează fiecare salt, nu mesajul; serverele de pe traseu îl citesc în clar. Doar criptarea capăt-la-capăt realizată de agenții utilizator (**S/MIME** sau **OpenPGP**) îl ascunde de servere. Împotriva **falsificării**: SMTP-ul simplu permite oricui să scrie orice `MAIL FROM` și `From:`; serverul destinatar verifică trei înregistrări publicate în **DNS-ul** domeniului expeditor:

Mecanism	Ce verifică destinatarul
SPF (RFC 7208)	o înregistrare TXT care enumeră adresele autorizate să trimită poștă pentru domeniu: `v=spf1 ip4:193.231.0.0/16 include:_spf.google.com -all`
DKIM (RFC 6376)	o semnătură digitală asupra antetului și corpului, adăugată de serverul expeditor într-un câmp `DKIM-Signature`; cheia publică se află în DNS
DMARC (RFC 7489)	o înregistrare de politică la `_dmarc.domain`: domeniul din `From:` trebuie să corespundă unei verificări SPF sau DKIM reușite, altfel mesajul este pus în carantină sau respins — și se trimit rapoarte

Din 2024, cei mai mari furnizori de cutii poștale resping corespondența în masă care nu trece aceste verificări. Filtrarea spamului adaugă peste acestea analiza conținutului și reputația.

6. Usenet: știri prin inundare

Usenet (1979–80; Tom Truscott și Jim Ellis la Duke, Steve Bellovin la UNC) este un sistem de discuții distribuit la nivel mondial, mai vechi decât Web-ul. Mesajele, numite **articole**, sunt publicate în **grupuri de știri (newsgroups)** organizate în ierarhii — „Big Eight”: `comp.`, `sci.`, `rec.`, `soc.`, `talk.`, `news.`, `misc.`, `humanities.`, plus `alt.` și multe ierarhii naționale și locale: `comp.os.linux.networking`, `sci.physics`, `ro.comp.retele`. Un articol are formatul unui mesaj de poștă (RFC 5536), cu câmpuri de antet suplimentare: `Newsgroups:`, un `Message-ID:` unic, `References:` (firul discuției) și `Path:`.

Usenet **nu are server central**. Serverul de știri al fiecărui site păstrează copii ale articolelor din grupurile pe care le găzduiește și schimbă articolele noi cu serverele vecine prin **inundare (flooding)**: un server oferă fiecare articol nou fiecărui partener, care îl acceptă doar dacă nu a mai văzut acel `Message-ID`. Fiecare server care retransmite articolul își adaugă propriul nume în stânga câmpului `Path:` (`Path: news.uni.ro!feed.example.net!origin.example.com!not-for-mail`) și nu îl oferă serverelor deja numite acolo. În câteva ore, un articol ajunge la mii de servere, pe orice legături există — o schemă de distribuție peer-to-peer cu decenii înainte de BitTorrent.

	Protocol și utilizare
NNTP (Network News Transfer Protocol, RFC 3977; portul 119, 563 peste TLS)	protocolul text atât pentru citire, cât și pentru transferul de la server la server, cu răspunsuri de trei cifre ca SMTP
Citire	`LIST` (grupurile), `GROUP comp.protocols.tcp-ip` (selectare; răspuns: numărul, primul și ultimul număr de articol), `OVER 100-120` (rezumate), `ARTICLE n`, `HEAD`, `BODY`, `POST` (urmat de articol, încheiat printr-un punct)
De la server la server	`IHAVE <message-id>` — „îl vrei?” (335 trimite-l / 435 nu este dorit); fluxurile continue (streaming feeds) folosesc `CHECK` și `TAKETHIS` (RFC 4644) pentru a evita câte un drum dus-întors pentru fiecare articol

Usenet a modelat cultura Internetului: Linus Torvalds a anunțat Linux pe `comp.os.minix` în 1991, Tim Berners-Lee a anunțat Web-ul pe `alt.hypertext` în același an, iar primul spam comercial a fost publicat în mii de grupuri de știri în 1994. Discuțiile s-au mutat între timp pe liste de discuții, forumuri web și rețele sociale — toate client–server — dar Usenet încă funcționează, iar proiectarea sa rămâne un caz clasic de distribuție prin inundare cu suprimarea duplicatelor, pe care îl vom reîntâlni la protocoalele de rutare.

7. DNS: servicii și ierarhie

Oamenii folosesc nume precum *www.unibuc.ro*; pachetele IP poartă adrese precum 141.85.x.x. **Sistemul de nume de domenii** (DNS; Paul Mockapetris, 1983; RFC-urile 1034 și 1035, 1987) este baza de date distribuită care face traducerea între ele. Este el însuși un protocol al nivelului aplicație, rulat de host-uri și de servere de nume, în mod normal peste **portul UDP 53** — o funcție esențială a Internetului implementată, fidel principiului capăt-la-capăt, la margine.

7.1 Ce oferă DNS

- **Nume către adresă**: înregistrări A (IPv4) și AAAA (IPv6).
- **Aliasuri**: o înregistrare CNAME dă numele canonic al unui alias (`www.shop.ro` → `shop.cdn-provider.net`).
- **Rutarea poștei**: înregistrările MX numesc serverele de poștă ale unui domeniu (secțiunea 3.2).
- **Distribuirea încărcării**: un nume solicitat intens poate avea mai multe adrese, returnate în ordine rotativă; CDN-urile returnează adrese diferite clienților aflați în locuri diferite.
- **Căutarea inversă**: înregistrările PTR asociază o adresă unui nume, sub `in-addr.arpa` (IPv4) și `ip6.arpa` (IPv6).
- **Înregistrări de text și de serviciu**: TXT (SPF, DMARC, verificarea domeniului), SRV (ce host și ce port oferă un serviciu), CAA (ce autorități de certificare pot emite certificate pentru domeniu).

7.2 De ce nu un singur server mare?

Un singur server ar fi un punct unic de defectare, ar primi interogările întregii planete, ar fi departe de majoritatea utilizatorilor și ar trebui actualizat pentru fiecare nume din lume. În schimb, DNS este o **ierarhie de zone**, fiecare administrată de proprietarul ei:

- zona **rădăcină**, servită de 13 identități numite de **servere rădăcină** (a.root-servers.net … m.root-servers.net), operate de 12 organizații și replicate prin anycast în mult peste o mie de locații; acestea cunosc serverele fiecărui domeniu de nivel superior;
- **domeniile de nivel superior** (TLD): generice (`.com`, `.org`, `.net` și sute de domenii noi), de cod de țară (`.ro`, administrat de ICI București ca RoTLD; `.de`, `.uk`) și `.arpa` pentru infrastructură; serverele lor cunosc serverele fiecărui domeniu de nivelul al doilea;
- **serverele autoritative** ale zonei fiecărei organizații (`unibuc.ro`), care dețin înregistrările propriu-zise. O zonă poate **delega** un subdomeniu (`fmi.unibuc.ro`) altor servere prin înregistrări NS.

Două roluri suplimentare completează imaginea. Fiecare host are un **resolver stub** — o bibliotecă ce trimite interogări. Iar fiecare rețea are **resolvere recursive** — operate de ISP sau de universitate, ori publice, precum 1.1.1.1, 8.8.8.8 și 9.9.9.9 — care fac munca de rezolvare pentru host-uri și păstrează rezultatele în cache.

8. DNS: rezolvare, cache, înregistrări și mesaje

8.1 Rezolvarea unui nume

Un host îi adresează resolverului local o interogare **recursivă**: „dă-mi răspunsul final”. Resolverul, dacă nu are nimic în cache, pune interogări **iterative**: serverul rădăcină răspunde cu o **trimitere (referral)** către serverele `.ro`; un server `.ro` răspunde cu o trimitere către serverele `unibuc.ro`, inclusiv adresele acestora (**înregistrări glue**) acolo unde este necesar; serverul autoritativ răspunde cu înregistrarea. Resolverul o returnează host-ului și păstrează în cache fiecare bucată.

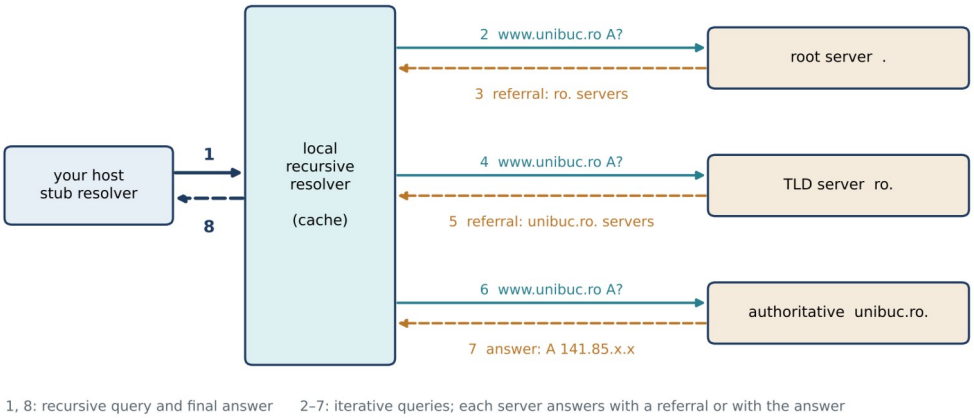


Figura 2. Rezolvarea numelui www.unibuc.ro cu un cache gol: o interogare recursivă de la host, trei interogări iterative de la resolver.

8.2 Cache-ul și TTL

Fiecare înregistrare poartă un **TTL** (time to live — durata de viață, în secunde), ales de proprietarul zonei: cât timp o pot păstra alții în cache. Cât timp durează, resolverul răspunde din cache, iar TTL-ul pe care îl raportează scade. Trimiterile de la TLD și de la rădăcină sunt și ele păstrate în cache — o zi sau două — astfel încât serverele rădăcină văd doar o mică parte din interogări. Răspunsurile negative („nume inexistent”) sunt de asemenea păstrate în cache, pentru un timp indicat în înregistrarea SOA a zonei. Cache-ul face DNS rapid și scalabil; prețul este că modificările se propagă încet: un administrator care vrea să mute un serviciu scade TTL-ul cu o zi înainte. Browserele și sistemele de operare își păstrează, în plus, propriile cache-uri mici.

8.3 Înregistrări de resurse

O zonă este un set de **înregistrări de resurse** (RR), fiecare fiind un tuplu *(nume, TTL, clasă, tip, valoare)* — clasa este aproape întotdeauna IN, Internet. O parte dintr-un fișier de zonă:

```
$ORIGIN example.ro.
@      3600  IN SOA   ns1.example.ro. admin.example.ro. (
                      2026102601 7200 900 1209600 300 )
@      86400 IN NS    ns1.example.ro.
@      86400 IN NS    ns2.provider.net.
ns1    86400 IN A     192.0.2.53
@      3600  IN MX    10 mail.example.ro.
mail   3600  IN A     192.0.2.25
www    300   IN A     192.0.2.80
www    300   IN AAAA  2001:db8::80
shop   3600  IN CNAME shops.cdn-provider.net.
@      3600  IN TXT   "v=spf1 mx -all"
```

Tip	Valoare
A / AAAA	o adresă IPv4 / IPv6
NS	numele unui server autoritativ pentru zonă (delegare)
CNAME	numele canonic al unui alias; resolverul continuă cu acesta
MX	o preferință și numele unui server de poștă
SOA	start of authority (începutul autorității): serverul primar, administratorul, numărul de serie, temporizatoarele de reîmprospătare și expirare, TTL-ul pentru cache-ul negativ
PTR, TXT, SRV, CAA	asociere inversă; text liber; localizarea serviciilor; autoritățile de certificare permise

Pentru a obține un domeniu, o organizație îl înregistrează printr-un **registrar**; **registrul (registry)** TLD-ului publică apoi în zona TLD înregistrările NS (și glue) ale serverelor autoritative ale organizației.

8.4 Mesajul DNS

Interogările și răspunsurile au un format comun: un **antet** de 12 octeți și patru secțiuni. Toate numerele sunt în ordinea octeților de rețea — veți construi singuri acești octeți în laborator.

Câmp	Dimensiune	Semnificație
ID	16 biți	ales de client, copiat în răspuns: asociază răspunsurile cu interogările
Indicatori (flags)	16 biți	QR (0 interogare, 1 răspuns), opcode, **AA** (răspuns autoritativ), **TC** (trunchiat), **RD** (recursivitate dorită), **RA** (recursivitate disponibilă), rcode (0 fără eroare, 2 eroare de server, 3 nume inexistent, 5 refuzat)
QDCOUNT, ANCOUNT, NSCOUNT, ARCOUNT	4 × 16 biți	numărul de intrări din cele patru secțiuni
Întrebare (Question)	variabilă	numele, sub formă de etichete (`3www6unibuc2ro0`), QTYPE și QCLASS
Răspuns, Autoritate, Suplimentar (Answer, Authority, Additional)	variabilă	înregistrări de resurse: răspunsul; înregistrările NS ale unei trimiteri; înregistrări suplimentare, precum adresele glue

Pentru a păstra răspunsurile mici, un nume care a apărut deja în mesaj este înlocuit printr-un **pointer** de 2 octeți — cu primii doi biți setați, iar ceilalți 14 dând offsetul copiei anterioare (`C0 0C` indică numele din întrebare, de la offsetul 12). Un răspuns UDP era inițial limitat la 512 octeți; **EDNS(0)** (RFC 6891) le permite clientului și serverului să convină asupra unor mesaje UDP mai mari, iar când un răspuns tot nu încape, serverul setează TC și clientul repetă interogarea peste **TCP**, care este folosit și pentru transferurile de zonă între servere.

8.5 Securitatea și confidențialitatea DNS

Un răspuns DNS simplu este acceptat dacă ID-ul și portul său corespund interogării; un atacator care le ghicește primul poate **otrăvi** cache-ul unui resolver, trimițându-i toți utilizatorii către o adresă falsă (atacul Kaminsky, 2008; porturile sursă aleatoare fac acum ghicirea mult mai grea). **DNSSEC** adaugă semnături digitale (înregistrări RRSIG, chei în înregistrări DNSKEY, un lanț de încredere prin înregistrări DS până la rădăcină), astfel încât un resolver să poată verifica **autenticitatea** unui răspuns; nu îl ascunde. **DNS peste TLS** (DoT, portul 853) și **DNS peste HTTPS** (DoH, RFC 8484)

criptează schimbul dintre host și resolver, astfel încât rețeaua Wi-Fi a cafenelei din Cursul 4 nu mai vede ce nume căutați. Fiind infrastructură critică, DNS este și o țintă: în 2016, atacul botnetului Mirai asupra furnizorului de DNS Dyn a făcut site-uri web majore inaccesibile timp de ore întregi.

9. NAT: multe host-uri în spatele unei singure adrese

O adresă IPv4 are 32 de biți, aproximativ 4,3 miliarde de valori, iar registrele regionale au rămas fără blocuri noi între 2011 și 2019. **Translatarea adreselor de rețea** (RFC 3022) permite unei întregi rețele să partajeze o singură adresă publică. În interior, host-urile folosesc **adrese private** (RFC 1918), pe care oricine le poate folosi și care nu sunt niciodată rutate în Internet: **10.0.0.0/8**, **172.16.0.0/12** și **192.168.0.0/16**.

9.1 Cum funcționează

Routerul NAT rescrie fiecare pachet care îl traversează. Pentru un pachet de ieșire, înlocuiește adresa și portul sursă private cu propria adresă publică și un port ales de el și înregistrează perechea în **tabela de translatare**; pentru un pachet de intrare, caută portul destinație în tabelă și rescrie destinația înapoi în adresa și portul private. Sumele de control sunt actualizate corespunzător. Deoarece sunt translatate și porturile, denumirea corectă este NAPT (network address and port translation — translatarea adreselor și porturilor de rețea); în practică, toată lumea spune NAT.

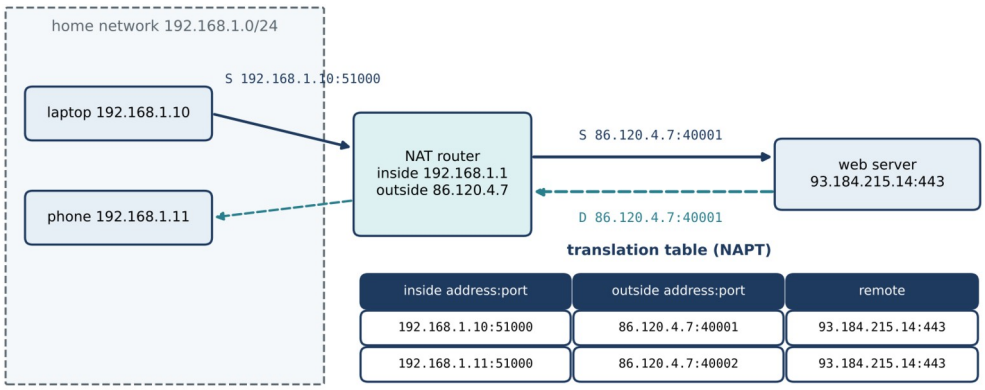


Figura 3. Două host-uri cu același port sursă ajung la același server printr-o singură adresă publică. Routerul le deosebește răspunsurile după portul pe care l-a atribuit fiecărei mapări.

9.2 Consecințe

- **Capacitate.** O adresă publică oferă aproximativ 64 000 de porturi pentru mapări, pentru fiecare protocol de transport (mai multe, dacă mapările sunt refolosite către destinații diferite). ISP-urile care duc lipsă de adrese folosesc **NAT la nivel de operator** (carrier-grade NAT, CGNAT), cu clienții în intervalul partajat 100.64.0.0/10, în spatele NAT-ului propriu al ISP-ului, și alocă fiecărui client un bloc de porturi.
- **Conexiunile de intrare sunt imposibile în mod implicit.** Un pachet care sosește pentru un port fără mapare nu are unde să meargă. Un server aflat în spatele NAT are nevoie de **redirecționarea porturilor (port forwarding)** — o mapare statică configurată pe router — sau de o mapare cerută de aplicație (UPnP IGD sau PCP, RFC 6887). Ca efect secundar, NAT blochează traficul nesolicitat, dar nu este un înlocuitor pentru un firewall.

- **Mapările expiră.** Mapările UDP inactive pot dispărea după zeci de secunde, cele TCP după minute sau ore; aplicațiile de lungă durată trimit mesaje **keepalive**.
- **Adresele din interiorul mesajelor nu mai funcționează.** FTP-ul activ (Cursul 4) scrie adresa privată a clientului în PORT; telefonia SIP scrie adrese în mesajele sale. Routerele includ **gateway-uri la nivel de aplicație** care rescriu astfel de mesaje — imposibil atunci când conexiunea de control este criptată.
- **Peer-to-peer are nevoie de ajutor.** Două host-uri aflate amândouă în spatele NAT nu se pot conecta pur și simplu. **STUN** (RFC 8489) permite unui host să-și afle adresa și portul public de la un server din exterior; ambii parteneri trimit apoi simultan pachete către maparea publică a celuilalt, deschizându-și NAT-urile (**hole punching**); când acest lucru eșuează, **TURN** (RFC 8656) retransmite traficul, iar **ICE** (RFC 8445) încearcă toate opțiunile în ordine — mecanismul din interiorul fiecărui apel video WebRTC.

NAT încalcă stratificarea din Cursul 2 — un router care rescrie antete de transport și chiar mesaje de aplicație — și principiul capăt-la-capăt, deoarece adresa unui host nu mai este cea pe care o văd partenerii săi. Cu toate acestea, a ținut în viață IPv4 timp de trei decenii. **IPv6**, cu 2^{128} adrese, îl face inutil: fiecare dispozitiv poate avea din nou o adresă unică la nivel global, protejată, acolo unde este nevoie, de un firewall. Vom reveni la NAT și IPv6 odată cu nivelul rețea.

Rezumat

- Poșta electronică este asincronă: **agenți utilizator**, **servere de poștă** cu cutii poștale și cozi, **SMTP** pentru împingere, **POP3/IMAP** pentru tragere, înregistrări **DNS MX** pentru găsirea serverului destinatar.
- **SMTP** este un protocol text de comenzi (EHLO, MAIL FROM, RCPT TO, DATA, QUIT) și răspunsuri de trei cifre; mesajul se termină la o linie cu un singur punct (dot-stuffing). **Plicul** (MAIL FROM, RCPT TO) este distinct de **antet** (From:, To:). Porturile 25 (retransmitere), 587 și 465 (trimitere).
- Un mesaj este format dintr-un **antet** și un **corp** (RFC 5322). **MIME** adaugă tipuri de conținut, corpuri multipart și codificări de transfer; **base64** crește datele binare cu aproximativ 37 %.
- **POP3** descarcă poșta dintr-o cutie poștală (stările AUTHORIZATION, TRANSACTION, UPDATE). **IMAP** păstrează poșta, folderele și marcajele pe server, cu comenzi etichetate, căutare pe server și preluare parțială. TLS protejează fiecare salt; **S/MIME** sau **OpenPGP** protejează mesajul capăt la capăt; **SPF**, **DKIM** și **DMARC** combat falsificarea.
- **Usenet** distribuie articolele din grupurile de știri între serverele de știri prin **inundare**, cu suprimarea duplicatelor după Message-ID și evitarea buclelor prin câmpul Path; **NNTP** deservește atât cititorii, cât și fluxurile de la server la server.
- **DNS** este o bază de date ierarhică, distribuită și cu cache: servere rădăcină, TLD și autoritative; resolvere recursive care efectuează interogări iterative; cache bazat pe **TTL**; înregistrări de resurse (A, AAAA, NS, CNAME, MX, SOA, PTR, TXT); mesaje cu un antet de 12 octeți și patru secțiuni, compresia numelor, portul UDP 53 cu recurgere la TCP; DNSSEC pentru autenticitate, DoT/DoH pentru confidențialitate.
- **NAT** mapează multe adrese private (RFC 1918) pe o singură adresă publică, rescriind adrese și porturi și păstrând o tabelă de translatare. Blochează conexiunile de intrare dacă porturile nu sunt redirecționate, strică protocoalele care transportă adrese și obligă aplicațiile peer-to-peer să folosească STUN, TURN și ICE. IPv6 elimină necesitatea lui.

Termeni-cheie

agent utilizator · server de poștă · cutie poștală · coadă de mesaje · server de trimitere · agent de transfer al poștei · SMTP · EHLO · MAIL FROM · RCPT TO · DATA · dot-stuffing · plic · antet · cale inversă · bounce · portul 25 / 587 / 465 · STARTTLS · înregistrare MX · RFC 5322 · Received · Message-ID · MIME · Content-Type · multipart · boundary · Content-Transfer-Encoding · base64 · quoted-printable · cuvânt codificat · POP3 · AUTHORIZATION / TRANSACTION / UPDATE · UIDL · IMAP · folder · marcaj (flag) · etichetă (tag) · răspuns neetichetat · IDLE · webmail · JMAP · S/MIME · OpenPGP · SPF · DKIM · DMARC · Usenet · grup de știri · articol · ierarhie · inundare · Path · NNTP · IHAVE · DNS · resolver stub · resolver recursiv · server rădăcină · TLD · server autoritativ · zonă · delegare · înregistrare glue · interogare recursivă / iterativă · trimitere (referral) · TTL · cache negativ · înregistrare de resursă · A · AAAA · NS · CNAME · MX · SOA · PTR · TXT · SRV · CAA · registrar · registru (registry) · antet DNS · QR · AA · TC · RD · RA · rcode · compresia numelor · EDNS(0) · otrăvirea cache-ului · DNSSEC · DoT · DoH · NAT · NAPT · adresă privată · RFC 1918 · tabelă de translatare · CGNAT · redirecționarea porturilor · UPnP · PCP · keepalive · gateway la nivel de aplicație · STUN · hole punching · TURN · ICE · IPv6

Întrebări de recapitulare

1. De ce trece un mesaj de poștă de obicei prin două servere de poștă în loc să meargă direct de pe calculatorul expeditorului pe cel al cititorului?
2. Ce comenzi SMTP formează plicul? Dați două situații în care destinatarul din plic nu apare în antetul `To:`.
3. De unde știe SMTP unde se termină un mesaj? Ce se întâmplă cu o linie a mesajului care începe cu un punct?
4. De ce trebuie codificate atașamentele binare și cu cât le crește base64 dimensiunea?
5. O utilizatoare își citește poșta pe telefon, pe laptop și în browser. De ce este IMAP mai bun decât POP3 pentru ea?
6. Ce protejează STARTTLS și ce nu protejează?
7. Cum evită un server Usenet să stocheze un articol de două ori și să îl trimită înapoi de unde a venit?
8. De ce este DNS distribuit și nu centralizat? Dați trei motive.
9. Care este diferența dintre o interogare recursivă și una iterativă? Ce servere răspund de obicei la fiecare?
10. De ce ar putea un site web care și-a schimbat adresa să rămână inaccesibil pentru unii utilizatori timp de ore întregi?
11. Ce înseamnă bitul TC dintr-un răspuns DNS și ce face clientul?
12. De ce nu poate un utilizator casnic să ruleze un server web accesibil din Internet fără să configureze routerul? Ce trebuie configurat?

Exerciții

1. Un mesaj conține 2 KB de text și un atașament PDF de 6 MB, codificat în base64 cu linii de 76 de caractere plus CR LF. Estimați dimensiunea mesajului pe fir (1 MB = 10^6 octeți).

2. RTT-ul unui host până la resolverul local este de 3 ms; RTT-urile resolverului până la un server rădăcină, un server TLD și un server autoritativ sunt 25 ms, 35 ms și 20 ms. Ignorați procesarea. Cât așteaptă host-ul adresa lui www.example.ro (a) cu un cache gol; (b) când resolverul are în cache trimiterea pentru .ro; (c) când are în cache răspunsul?
3. Scrieți octeții secțiunii de întrebare a unei interogări DNS pentru înregistrarea AAAA a lui `ftp.unibuc.ro`.
4. Un router NAT cu adresa publică 86.120.4.7 primește un pachet de la 192.168.1.20:50000 către 1.1.1.1:53 (UDP), apoi unul de la 192.168.1.21:50000 către 1.1.1.1:53. Scrieți pachetele translatate și tabela de translatare, presupunând că routerul alocă porturi începând cu 61000.
5. Un ISP cu 40 000 de clienți folosește CGNAT și îi dă fiecărui client un bloc de 2 000 de porturi pe adresă publică. De câte adrese IPv4 publice are nevoie (folosiți 64 000 de porturi utilizabile pe adresă)?

Răspunsuri

1. 6 000 000 de octeți → 8 000 000 de caractere base64; 8 000 000 / 76 ≈ 105 264 de linii, adăugând câte 2 octeți ≈ 210 528 de octeți; plus aproximativ 2 KB de text și antetele: **≈ 8,2 MB** — cu 37 % mai mult decât atașamentul.
2. (a) 3 + 25 + 35 + 20 = **83 ms**. (b) 3 + 20 = **23 ms**. (c) **3 ms** — și 0 ms dacă îl are cache-ul propriu al host-ului.
3. `03 66 74 70 06 75 6e 69 62 75 63 02 72 6f 00` (etichetele ftp, unibuc, ro și rădăcina), apoi QTYPE `00 1c` (28 = AAAA) și QCLASS `00 01` (IN): 19 octeți.
4. Pachetul 1 devine 86.120.4.7:61000 → 1.1.1.1:53; pachetul 2 devine 86.120.4.7:61001 → 1.1.1.1:53. Tabela: 192.168.1.20:50000 ↔ 86.120.4.7:61000 și 192.168.1.21:50000 ↔ 86.120.4.7:61001 (UDP, la distanță 1.1.1.1:53). Răspunsurile către 61000 și 61001 sunt translatate înapoi către host-ul corect, deși ambele host-uri au folosit același port sursă.
5. 64 000 / 2 000 = 32 de clienți pe adresă; 40 000 / 32 = **1 250 de adrese publice** în loc de 40 000.

Lecturi și pașii următori

Obligatoriu

- Kurose și Ross, *Computer Networking: A Top-Down Approach*, ediția a 8-a, Capitolul 2: poșta electronică în Internet; DNS — serviciul de director al Internetului. Capitolul 4: secțiunea despre NAT.

Recomandat

- RFC 5321 (SMTP), secțiunile 2–4; RFC 5322 (formatul mesajelor); RFC 2045 (MIME); RFC 1939 (POP3, scurt și ușor de citit); RFC 9051 (IMAP4rev2), secțiunea 2.
- RFC 3977 (NNTP), secțiunile 3–6; RFC 1034 (conceptele DNS), secțiunile 2–5; RFC 1035, secțiunea 4 (formatul mesajelor).
- RFC 3022 (NAT tradițional); RFC 4787 (comportamentul NAT pentru UDP); RFC 8445 (ICE), secțiunea 2.

Laboratorul 5 (săptămâna aceasta)

Trimiteți poștă tastând SMTP și cu un atașament MIME, o citiți cu POP3, priviți în interiorul unui mesaj de întâmpinare IMAP, urmăriți rezolvarea DNS cu dig, construiți interogări DNS octet cu octet

peste socket-uri UDP și construiți propriul router NAT din spații de nume de rețea (network namespaces) Linux — sau, pe Windows, găsiți NAT-urile dintre dumneavoastră și Internet.

Cursul următor

Nivelul transport I: multiplexarea și demultiplexarea, UDP și principiile transferului fiabil de date.

PARTEA B · SĂPTĂMÂNA 5

Seminarul 5: mesaje, nume și adrese translatate

Sesiune de probleme de două ore (2 × 50 de minute) care însoțește Cursul 5

Seminarul transformă Cursul 5 în citirea atentă a dialogurilor și octeților de protocol, mici calcule și proiecte. Lucrați în perechi; câte o pereche prezintă fiecare problemă.

La sfârșitul seminarului ar trebui să puteți

- citi un dialog SMTP și separa plicul de antet;
- estima dimensiunea unui mesaj MIME și citi traseul unui mesaj din câmpurile sale Received;
- alege între POP3 și IMAP și citi un schimb IMAP;
- urmări inundarea unui articol între serverele Usenet;
- calcula timpii de rezolvare DNS cu și fără cache, decodifica octet cu octet un răspuns DNS și scrie o zonă mică;
- completa o tabelă de translatare NAT și raționa despre capacitatea NAT, redirecționarea porturilor și traficul peer-to-peer.

Plan

Pas	Activitate	Timp
A	Problemele 1–2: un dialog SMTP; dimensiunea și traseul unui mesaj	17 min
B	Problemele 3–4: POP3 sau IMAP; citirea IMAP	13 min
C	Problema 5: inundarea știrilor	10 min
	Pauză	10 min
D	Problemele 6–8: DNS — timpi, octeți și o zonă	25 min
E	Problemele 9–11: tabele NAT, capacitate și traversare	25 min
	Temă	(în E)

A. Poșta electronică pe fir

Problema 1

Un server a înregistrat următoarea sesiune (C client, S server):

```
S: 220 mx.example.com ESMTP
C: EHLO mail.uni.ro
S: 250-mx.example.com
S: 250-SIZE 26214400
S: 250 STARTTLS
C: MAIL FROM:<list-bounces@uni.ro>
S: 250 OK
C: RCPT TO:<bob@example.com>
S: 250 OK
C: RCPT TO:<carol@example.com>
S: 550 5.1.1 No such user
C: RCPT TO:<dan@example.com>
S: 250 OK
C: DATA
S: 354 Go ahead
```

```
C: From: Prof. Ionescu <ionescu@uni.ro>
C: To: Students <cn-2026@uni.ro>
C: Subject: Exam room
C:
C: The exam is in room 105.
C: ..and bring your student card.
C: .
S: 250 OK queued as 91BC
C: QUIT
S: 221 Bye
```

1. Cine primește mesajul pe acest server? Ce va afla expeditorul despre carol și de la cine?
2. Niciunul dintre destinatari nu apare în câmpul `To:`. Explicați de ce mesajul este totuși livrat. Numiți cele două adrese din plic și cele două din antet.
3. Care este ultima linie a corpului, așa cum o citește Bob?
4. Ce pas important lipsește între EHLO și MAIL FROM, având în vedere extensiile serverului? Ce ar putea citi un observator de pe traseu?

Soluție

1. Bob și Dan (ambele RCPT au primit răspunsul 250). Carol a fost refuzată cu o eroare permanentă (550) în timpul sesiunii, astfel încât serverul *expeditor* — mail.uni.ro — generează un bounce către calea inversă `list-bounces@uni.ro`, adică către software-ul listei, nu către profesor.

2. Livrarea folosește doar plicul: `MAIL FROM:<list-bounces@uni.ro>` și adresele din RCPT TO. Antetele `From: ionescu@uni.ro` și `To: cn-2026@uni.ro` sunt doar afișate. Așa funcționează listele de discuții (și Bcc).

3. `.and bring your student card.` — linia începea cu un punct, așa că clientul l-a dublat (dot-stuffing), iar serverul elimină un punct.

4. STARTTLS (apoi din nou EHLO în interiorul TLS). Fără el, oricine de pe traseu citește întregul dialog: adresele, subiectul și corpul.

Problema 2

Bob primește un mesaj cu o fotografie de 4,5 MB atașată (base64, 76 de caractere pe linie plus CR LF) și 1 KB de text. Antetul său începe astfel:

```
Received: from mx1.example.com (mx1.example.com [198.51.100.7])
    by imap.example.com with LMTP; Mon, 26 Oct 2026 09:14:05 +0200
Received: from mail.uni.ro (mail.uni.ro [193.231.20.10])
    by mx1.example.com with ESMTPS; Mon, 26 Oct 2026 09:14:03 +0200
Received: from [192.168.1.34] (unknown [86.120.4.7])
    by mail.uni.ro with ESMTPSA; Mon, 26 Oct 2026 09:13:58 +0200
From: Alice <alice@uni.ro>
```

1. Estimați dimensiunea mesajului (1 MB = 10^6 octeți). Îl va accepta serverul din Problema 1?
2. Enumerați în ordine salturile mesajului, de pe calculatorul lui Alice până în cutia poștală a lui Bob. Ce salt a durat cel mai mult?
3. Ce vă spun ESMTPSA și cele două adrese din ultimul câmp `Received` despre Alice?

Soluție

1. 4 500 000 × 4/3 = 6 000 000 de caractere; 6 000 000 / 76 ≈ 78 948 de linii × 2 octeți ≈ 158 kB de terminatori de linie; total ≈ **6,16 MB** plus câțiva kB de text și antet. Limita SIZE de 26 214 400 de octeți (25 MiB) nu este depășită.
2. Citiți de jos în sus: calculatorul lui Alice → mail.uni.ro (09:13:58) → mx1.example.com (09:14:03) → imap.example.com, serverul cutiei poștale a lui Bob (09:14:05). Saltul de retransmitere de la mail.uni.ro la mx1.example.com a durat cel mai mult (5 s, inclusiv așteptarea în coadă și verificările antispam); ceasurile diferitelor servere pot diferi ușor, deci astfel de timpi sunt doar orientativi.
3. ESMTP cu **S**ecuritate (TLS) și **A**utentificare: Alice a trimis mesajul după ce s-a autentificat. Calculatorul ei s-a prezentat ca 192.168.1.34 — o adresă privată — dar conexiunea a venit de la 86.120.4.7: ea se află în spatele unui router NAT (secțiunea 9 a cursului).

B. Citirea poștei

Problema 3

Pentru fiecare utilizator, spuneți dacă este mai potrivit POP3 sau IMAP și de ce:

1. un cercetător aflat pe o navă, cu o legătură prin satelit timp de o oră pe zi, care își citește poșta doar pe un singur laptop;
2. un manager care își citește poșta pe telefon, pe tabletă și pe PC-ul de la birou și care arhivează mesajele în foldere;
3. sistemul de comenzi al unui magazin, care colectează în fiecare minut confirmările de comandă dintr-o cutie poștală și îl procesează pe fiecare o singură dată;
4. un student care vrea să citească subiectele a 3 000 de mesaje printr-o legătură mobilă lentă, apoi doar două dintre mesaje.

Soluție

1. **POP3**: descarcă totul în fereastra de conectare, citește și păstrează offline; nimic nu trebuie să rămână sincronizat cu alte dispozitive.
2. **IMAP**: folderele, marcajele de citire și ștergerile sunt păstrate pe server și văzute identic de pe toate cele trei dispozitive.
3. **POP3** este suficient (sau IMAP): `UIDL` îi dă fiecărui mesaj un identificator stabil, astfel încât programul preia fiecare mesaj o singură dată, apoi îl șterge cu `DELE`.
4. **IMAP**: `FETCH 1:3000 (BODY.PEEK[HEADER.FIELDS (SUBJECT)])` transferă doar subiectele; cele două mesaje sunt apoi preluate integral. Cu POP3 este posibil `TOP n 0` pentru fiecare mesaj, dar necesită 3 000 de comenzi și nu oferă căutare pe server sau foldere.

Problema 4

```
C: t1 SELECT INBOX
S: * 12 EXISTS
S: * OK [UIDNEXT 4392] Predicted next UID
S: t1 OK [READ-WRITE] SELECT completed
C: t2 SEARCH UNSEEN FROM "ionescu"
S: * SEARCH 9 12
S: t2 OK SEARCH completed
C: t3 FETCH 12 (RFC822.SIZE BODY.PEEK[HEADER.FIELDS (SUBJECT)])
S: * 12 FETCH (RFC822.SIZE 6158210 BODY[HEADER.FIELDS (SUBJECT)] {22}
S: Subject: Exam room
S: )
S: t3 OK FETCH completed
C: t4 STORE 9 +FLAGS (\Deleted)
C: t5 EXPUNGE
```

1. Câte mesaje sunt în folder și ce mesaje necitite provin de la Ionescu? Unde s-a făcut această căutare?
2. De ce `BODY.PEEK` și nu `BODY`? Ce anunță `{22}`?
3. Mesajul 12 este mare. Ce ar face în continuare un client mobil atent?
4. Ce se întâmplă cu mesajul 9 după t4? Dar după t5?

> **Soluție**
> 1. 12 mesaje; 9 și 12 sunt necitite și provin de la Ionescu. Căutarea s-a executat **pe server**, deci niciun mesaj nu a traversat rețeaua.
> 2. `BODY.PEEK` nu setează marcajul `\Seen`, astfel încât mesajul rămâne necitit pe toate dispozitivele. `{22}` este un prefix de lungime: urmează 22 de octeți de date literale (linia de subiect și CR LF, linia goală) înaintea parantezei de închidere.
> 3. Să nu descarce 6 MB: să preia structura (`BODYSTRUCTURE`), să afișeze doar partea de text (`BODY[1]`) și să preia atașamentul când utilizatorul îl cere.
> 4. După t4 este doar marcat `\Deleted` (încă prezent, încă vizibil pentru alți clienți); `EXPUNGE` îl elimină definitiv — echivalentul IMAP al stării UPDATE din POP3.

C. Inundarea știrilor

Problema 5

Cinci servere de știri sunt conectate astfel: A–B, A–C, B–C, B–D, C–D, D–E. Un articol este publicat pe A la momentul 0. Fiecare server oferă fiecare articol nou tuturor vecinilor săi, cu excepția celor deja numiți în câmpul `Path:` al articolului; o ofertă și un transfer pe o legătură durează 1 minut. Un server acceptă un articol doar dacă nu i-a văzut Message-ID-ul.

1. La ce moment primește fiecare server articolul pentru prima dată? Scrieți câmpul `Path:` așa cum este stocat pe E.
2. Câte transferuri sunt oferite în total și câte sunt refuzate ca duplicate?
3. De ce nu previne singur câmpul Path duplicatele și de ce este suficientă doar verificarea Message-ID pentru a opri inundarea?

Soluție

1. A: 0. B și C: 1 (de la A). D: 2 (de la B sau C — amândouă îl oferă în al doilea minut; să spunem că B este primul). E: 3 (de la D). Fiecare server își adaugă numele în față când acceptă articolul, astfel încât E stochează `Path: E!D!B!A!not-for-mail`.
2. A îl oferă lui B și lui C (2 acceptate). B (Path B!A) îl oferă lui C și lui D; C (Path C!A) îl oferă lui B și lui D: B→C și C→B sunt refuzate, B→D este acceptată, C→D este refuzată. D (Path D!B!A) îl oferă lui C (refuzat) și lui E (acceptat). E nu are alt vecin. Total **8 oferte: 4 acceptate (câte una pentru fiecare server în afară de A), 4 refuzate**. Cu IHAVE, un refuz costă doar un schimb scurt, nu articolul.
3. Path enumeră doar serverele de pe traseul *acestei copii*; copia lui B și copia lui C urmează trasee diferite și niciuna nu îl numește pe celălalt server. Verificarea Message-ID face ca fiecare server să accepte articolul o singură dată, deci fiecare server îl propagă o singură dată: munca totală este limitată de numărul de legături.

D. DNS

Problema 6

RTT-ul unui laptop până la resolverul facultății este de 2 ms. RTT-urile resolverului: rădăcina 20 ms, serverele `.ro` 30 ms, serverele `unibuc.ro` 15 ms, serverele `.com` 25 ms, serverele `cdn-provider.com` 40 ms. Înregistrări: `www.unibuc.ro` este un CNAME pentru `unibuc.cdn-provider.com` (TTL 3600), care are o înregistrare A cu TTL 60. Trimiterile TLD sunt păstrate în cache 2 zile. Ignorați procesarea și TCP.

1. Resolverul tocmai a pornit (cache gol). Cât durează până când laptopul are adresa lui www.unibuc.ro?
2. Un al doilea laptop cere același nume 30 s mai târziu; un al treilea, 5 minute mai târziu. Cât așteaptă fiecare?
3. De ce ar putea alege CDN-ul un TTL atât de scurt pentru înregistrarea A?

Soluție

1. Rădăcina (20) → serverul `.ro` (30) → serverul `unibuc.ro` (15), care returnează CNAME-ul. Resolverul trebuie acum să rezolve numele canonic sub `.com`: din nou rădăcina (20, deoarece doar trimiterea pentru `.ro` este în cache), apoi un server `.com` (25), apoi un server `cdn-provider.com` (40). Total = 2 + 20 + 30 + 15 + 20 + 25 + 40 = **152 ms**. (Resolverele reale au de obicei în cache zona rădăcină și TLD-urile populare cu mult timp înainte.)
2. După 30 s totul este în cache: **2 ms**. După 5 minute, CNAME-ul (TTL 3600) și trimiterea pentru `.com` sunt în cache, dar înregistrarea A (TTL 60) a expirat: o interogare către `cdn-provider.com`: 2 + 40 = **42 ms**.
3. Pentru a-și putea schimba repede răspunsul — pentru a direcționa clienții către un server mai apropiat sau mai puțin încărcat, ori departe de unul căzut. Prețul este un număr mai mare de interogări către serverele sale și o primă vizită mai lentă.

Problema 7

Un răspuns capturat pe portul UDP 53 (offseturile în hexazecimal, în stânga):

```
0000  1a 2b 81 80 00 01 00 02 00 00 00 00 03 77 77 77
0010  07 65 78 61 6d 70 6c 65 02 72 6f 00 00 01 00 01
0020  c0 0c 00 05 00 01 00 00 01 2c 00 06 03 77 65 62
0030  c0 10 c0 2c 00 01 00 01 00 00 01 2c 00 04 c0 00
0040  02 50
```

1. Decodificați antetul: ID-ul, fiecare indicator, cele patru numărători.
2. Decodificați întrebarea.
3. Decodificați cele două răspunsuri, rezolvând fiecare pointer de compresie. Care este adresa și sub ce nume?
4. Câți octeți a economisit compresia?

> **Soluție**
>
> 1. ID 0x1a2b. Indicatori 0x8180 = 1000 0001 1000 0000: QR = 1 (răspuns), opcode 0, AA = 0 (neautoritativ — a venit din cache-ul unui resolver sau prin recursivitate), TC = 0, RD = 1 (copiat din interogare), RA = 1, rcode 0 (fără eroare). QDCOUNT 1, ANCOUNT 2, NSCOUNT 0, ARCOUNT 0.
>
> 2. De la offsetul 0x0c: `03 www 07 example 02 ro 00` = www.example.ro; QTYPE 0x0001 = A, QCLASS 0x0001 = IN.
>
> 3. Răspunsul 1 la 0x20: numele `c0 0c` → offsetul 12 = www.example.ro; tipul 5 = **CNAME**, clasa IN, TTL 0x12c = 300 s, RDLENGTH 6; datele `03 web c0 10` = web + pointer către offsetul 0x10 (example.ro) = **web.example.ro**. Răspunsul 2 la 0x32: numele `c0 2c` → offsetul 0x2c, unde începe `03 web` = web.example.ro; tipul 1 = **A**, clasa IN, TTL 300, RDLENGTH 4, datele `c0 00 02 50` = **192.0.2.80**. Așadar www.example.ro este un alias al lui web.example.ro, a cărui adresă este 192.0.2.80.
>
> 4. Fără pointeri: numele răspunsului 1 ar avea 16 octeți în loc de 2 (14 economisiți), datele sale 16 în loc de 6 (10 economisiți — `web` + `example.ro` scrise integral), numele răspunsului 2 16 în loc de 2 (14 economisiți): **38 de octeți** economisiți, o treime din mesajul necomprimat de 104 octeți.

Problema 8

Administrați domeniul `retele-lab.ro`. Aveți două servere de nume (193.0.2.1 și unul la un furnizor de găzduire, `ns.host.net`), un server web la 193.0.2.80 care trebuie să răspundă și la `retele-lab.ro`, și la `www.retele-lab.ro`, un server de poștă la 193.0.2.25 cu o rezervă la furnizor (`mx.host.net`), iar doar serverul dumneavoastră de poștă are voie să trimită poștă pentru domeniu.

1. Scrieți înregistrările zonei (TTL-uri la alegere; omiteți SOA).
2. Ce trebuie să publice registrul `.ro` și de ce este necesară o înregistrare glue pentru un server, dar nu și pentru celălalt?
3. Veți muta serverul web la o adresă nouă vinerea viitoare. Ce faceți joi?

Soluție

1. O zonă posibilă:
`@ 86400 IN NS ns1.retele-lab.ro.` · `@ 86400 IN NS ns.host.net.` · `ns1 86400 IN A 193.0.2.1` · `@ 3600 IN A 193.0.2.80` · `www 3600 IN CNAME retele-lab.ro.` · `@ 3600 IN MX 10 mail.retele-lab.ro.` · `@ 3600 IN MX 20 mx.host.net.` · `mail 3600 IN A 193.0.2.25` · `@ 3600 IN TXT "v=spf1 ip4:193.0.2.25 -all"`. (Vârful zonei @ nu poate fi un CNAME, deoarece are deja înregistrări NS și MX; www poate.)
2. Înregistrările NS ale lui `retele-lab.ro` (ns1.retele-lab.ro și ns.host.net) în zona `.ro` și înregistrarea A a lui `ns1.retele-lab.ro` ca **glue**: altfel, un resolver nu ar putea afla adresa lui ns1 — aceasta se află în interiorul zonei pe care o servește. `ns.host.net` este rezolvat prin `.net`, deci nu are nevoie de glue.
3. Scădeți TTL-ul înregistrării A (de ex. la 60 s) cu cel puțin un TTL vechi (1 oră) înainte de mutare, astfel încât cache-urile să păstreze adresa veche cel mult un minut după schimbarea de vineri; apoi creșteți-l la loc.

E. NAT

Problema 9

Un router de acasă cu adresa publică 86.120.4.7 și LAN-ul 192.168.1.0/24 folosește NAT cu păstrarea portului: păstrează portul sursă când acesta este liber pe partea publică, altfel alege următorul port liber începând cu 40000. În ordine:

1. 192.168.1.10:51000 → 142.250.1.1:443 (TCP)
2. 192.168.1.11:51000 → 142.250.1.1:443 (TCP)
3. 192.168.1.11:5353 → 1.1.1.1:53 (UDP)
4. 142.250.1.1:443 → 86.120.4.7:40000 (TCP)
5. 203.0.113.9:22 → 86.120.4.7:22 (TCP SYN)

Completați tabela de translatare și dați fiecare pachet după translatare. Apoi routerul este configurat să redirecționeze portul TCP 2222 către 192.168.1.10:22: cum arată o conexiune de la 203.0.113.9:60000 către 86.120.4.7:2222 de ambele părți?

Soluție

1. 86.120.4.7:51000 → 142.250.1.1:443. Intrare: TCP 192.168.1.10:51000 ↔ 86.120.4.7:51000.
2. Portul 51000 este ocupat (către aceeași destinație): 86.120.4.7:40000 → 142.250.1.1:443. Intrare: TCP 192.168.1.11:51000 ↔ 86.120.4.7:40000.
3. UDP are o tabelă separată: 86.120.4.7:5353 → 1.1.1.1:53. Intrare: UDP 192.168.1.11:5353 ↔ 86.120.4.7:5353.
4. Corespunde intrării 2: livrat ca 142.250.1.1:443 → 192.168.1.11:51000.
5. Nicio intrare pentru portul 22: routerul aruncă pachetul (sau răspunde cu un TCP RST): conexiunile de intrare sunt imposibile fără redirecționarea porturilor.

Cu redirecționare: în exterior, 203.0.113.9:60000 → 86.120.4.7:2222; în interior, 203.0.113.9:60000 → 192.168.1.10:22. Serverul SSH vede adresa reală a clientului; răspunsurile sunt translatate înapoi de la 192.168.1.10:22 la 86.120.4.7:2222.

Problema 10

1. Un ISP are 2 048 de adrese IPv4 publice și 100 000 de clienți în spatele CGNAT. Folosind 64 000 de porturi utilizabile pe adresă, câte porturi poate avea fiecare client? Un browser deschide aproximativ 50 de conexiuni TCP pentru o pagină complexă, iar un joc online menține 20 de fluxuri UDP. Este alocarea suficientă pentru o familie de patru persoane?
2. O aplicație de mesagerie ține deschisă o conexiune TCP inactivă pentru a primi notificări. CGNAT-ul elimină mapările TCP inactive după 10 minute, un router de acasă după 2 ore. Cât de des trebuie să trimită aplicația un keepalive și ce se întâmplă altfel?
3. De ce nu are un client CGNAT nicio posibilitate de a redirecționa un port?

> **Soluție**
>
> 1. 2 048 × 64 000 / 100 000 ≈ **1 310 porturi pe client**. Patru persoane cu câteva browsere (fiecare filă având poate 20–50 de conexiuni), telefoane, jocuri și actualizări pot ajunge la câteva sute de mapări simultane: suficient în utilizare normală, deși utilizatorii intensivi pot rămâne fără porturi, ceea ce se manifestă prin pagini încărcate parțial. Din acest motiv ISP-urile alocă blocurile dinamic sau folosesc blocuri mai mari.
> 2. Mai des decât cel mai scurt timp de expirare de pe traseu — de ex. la fiecare 5–9 minute. Altfel, CGNAT-ul elimină maparea în tăcere; următoarea notificare a serverului sosește pentru un port fără mapare și se pierde, iar aplicația descoperă acest lucru abia când trimite din nou.
> 3. Maparea este creată pe NAT-ul ISP-ului, pe care clientul nu îl controlează: redirecționarea porturilor de pe routerul propriu de acasă ajunge doar la partea privată a clientului. Clientul are nevoie de o adresă IPv4 publică de la ISP (adesea contra cost), de IPv6 sau de un serviciu de retransmitere ori de tunelare.

Problema 11

Alice și Bob, fiecare în spatele unui NAT de acasă, vor să facă un apel video. Calculatoarele lor își cunosc doar adresele private.

1. De ce nu poate niciunul să trimită pur și simplu pachete UDP către adresa privată a celuilalt? Dar către adresa publică a celuilalt?
2. Descrieți cum le permit STUN și hole punching să schimbe pachete direct. Ce proprietate trebuie să aibă NAT-urile?
3. Când trebuie apelul să treacă prin TURN și ce costă acest lucru?

Soluție

1. Adresele private nu sunt rutate în Internet. Un pachet trimis către adresa *publică* a celuilalt ajunge la un NAT care nu are nicio mapare pentru el și este aruncat.
2. Fiecare trimite o cerere STUN Binding către un server și își află adresa și portul public (maparea proprie). Și le schimbă prin serverul de semnalizare (serverul aplicației, pe conexiunea existentă). Apoi amândoi trimit pachete UDP către maparea publică a celuilalt cam în același timp: pachetul de ieșire al lui Alice creează în NAT-ul ei o mapare care acceptă acum pachetele lui Bob și invers. Funcționează dacă NAT-urile sunt **independente de punctul final (endpoint-independent)**: folosesc același port public pentru același socket privat, indiferent de destinație (RFC 4787), astfel încât adresa aflată prin STUN este cea pe care trebuie să o folosească partenerul.
3. Când un NAT dă un port public diferit pentru fiecare destinație (NAT simetric) sau un firewall blochează UDP: un server TURN din exterior retransmite tot conținutul media. Costul: întârziere mai mare, iar operatorul plătește lățimea de bandă a releului. ICE încearcă mai întâi candidații direcți și recurge la TURN.

Temă

1. Găsiți un mesaj în propria cutie poștală (în Gmail: ⋮ → *Show original* / Afișați originalul). Copiați câmpurile sale `Received` și rezultatele SPF, DKIM și DMARC (`Authentication-Results`). Desenați traseul mesajului și spuneți ce server a verificat ce.
2. Cu `dig +norecurse`, rezolvați manual `www.unibuc.ro`, pornind de la `a.root-servers.net` și urmând fiecare trimitere. Arătați fiecare comandă și secțiunea răspunsului care v-a trimis mai departe (Authority / Additional).
3. Pe o foaie de hârtie, scrieți interogarea DNS completă (antet și întrebare, în hexazecimal) pentru înregistrarea MX a lui `unibuc.ro`, cu ID-ul 0x0042 și recursivitate dorită. Verificați-o cu tcpdump, comparând-o cu clientul dumneavoastră în C din Laboratorul 5.
4. Explicați în jumătate de pagină de ce un router NAT trebuie să rescrie comanda PORT a FTP-ului activ, dar nu poate face acest lucru pentru FTPS, și ce schimbă modul pasiv din Laboratorul 4.

PARTEA C · SĂPTĂMÂNA 5

Laboratorul 5 (Linux): poștă de mână, DNS octet cu octet, un router NAT propriu

Fișă de laborator de două ore (2 × 50 de minute) care însoțește Cursul 5

Săptămâna aceasta vorbiți chiar dumneavoastră SMTP și POP3 cu servere mici de pe propria mașină, priviți mesajul de întâmpinare al unui server IMAP real, urmăriți DNS cu dig și cu un client DNS scris în C peste socket-uri UDP și construiți o rețea de acasă cu un router NAT în interiorul unei singure mașini Linux, cu ajutorul spațiilor de nume de rețea. Toate programele au fost testate pe Ubuntu 24.04 și sunt furnizate ca fișiere.

> **La sfârșitul laboratorului ar trebui să puteți**
> • trimite un mesaj tastând SMTP și vedea la lucru dot-stuffing-ul și plicul;
> • trimite un mesaj MIME cu un atașament și citi forma sa codificată;
> • citi și șterge manual poșta cu POP3 și deschide o sesiune IMAP peste TLS;
> • urmări cu dig rezolvarea DNS, tipurile de înregistrări și TTL-urile și interoga DNS peste HTTPS;
> • construi o interogare DNS în C, o trimite peste UDP și decodifica răspunsul, inclusiv numele comprimate;
> • construi un router NAT, urmări tabela sa de translatare, configura redirecționarea porturilor și afla propria mapare publică cu STUN.

Înainte de a începe

- **Mediul de lucru.** Mașina virtuală a cursului sau orice Linux pe care aveți sudo (Sarcina 6 creează spații de nume de rețea). Instalați instrumentele o singură dată (pe Ubuntu, `python3-aiosmtpd` se află în depozitul universe; în altă parte `pip install aiosmtpd`):

```
sudo apt install -y build-essential dnsutils tcpdump curl netcat-openbsd \
    nftables conntrack python3-aiosmtpd
```

- **Fișiere.** Dezarhivați arhiva acestui laborator în `lab5/`: `linux/` (dns_query.c, Makefile, nat_lab.sh), `python/` (talk.py, send_mail.py, pop3_server.py, dns_query.py, stun_client.py) și `windows/`. Compilați cu `make -C linux`.
- **talk.py** este un mic terminal pentru protocoale text: fiecare linie pe care o tastați este trimisă cu CR LF, fiecare linie primită este afișată după `<-`. `nc -C host port` face același lucru, fără TLS.
- **Etică.** Serverele de poștă din Sarcinile 1–3 sunt ale dumneavoastră. Nu trimiteți niciodată manual poștă prin serverul altcuiva și nu încercați niciodată parole pe conturi care nu vă aparțin.
- **Ce predați.** Un scurt raport cu răspunsurile la **Q1–Q16** și ieșirile care le susțin; fișierul `dns_query.c` extins de dumneavoastră.

Plan

Pas	Activitate	Timp
1	SMTP de mână: un server de poștă propriu	15 min

Pas	Activitate	Timp
2	MIME: un atașament pe fir	10 min
3	Citirea poștei: POP3 de mână, mesajul de întâmpinare IMAP	15 min
	Pauză	10 min
4	DNS cu dig: rezolvare, înregistrări, TTL, DoH	15 min
5	Un client DNS în C peste UDP	20 min
6	Un router NAT propriu; STUN	15 min

Sarcina 1 — SMTP de mână

`aiosmtpd` este un mic server SMTP scris în Python. Cu handlerul `Mailbox`, stochează fiecare mesaj acceptat ca un fișier separat într-un **Maildir** — formatul de cutie poștală în care fiecare mesaj este un fișier în `new/`.

```
cd lab5
# terminalul A: un server SMTP pe portul 8025, care stochează poșta în ~/Maildir
python3 -m aiosmtpd -n -l 127.0.0.1:8025 -c aiosmtpd.handlers.Mailbox ~/Maildir
# terminalul B: vorbiți SMTP
python3 python/talk.py 127.0.0.1 8025          # sau: nc -C 127.0.0.1 8025
```

Tastați următoarele linii una câte una, așteptând fiecare răspuns (liniile care încep cu `<-` sunt ale serverului):

```
EHLO client.lab.test
MAIL FROM:<alice@lab.test>
RCPT TO:<bob@lab.test>
RCPT TO:<carol@lab.test>
DATA
From: Alice <alice@lab.test>
To: Bob <bob@lab.test>
Subject: typed by hand

Hello Bob,
..this line started with one dot.
.
QUIT
```

```
ls ~/Maildir/new/                    # câte un fișier pentru fiecare mesaj
cat ~/Maildir/new/*                  # antetul adăugat de server, apoi al dumneavoastră
```

- **Q1.** Copiați răspunsul serverului la EHLO. Ce extensii anunță? Ce coduri de răspuns ați primit pentru MAIL, RCPT, DATA, punctul final și QUIT și din ce clasă (2xx/3xx/4xx/5xx) face parte fiecare?
- **Q2.** Carol nu este numită în nicio linie de antet. Primește ea mesajul? Ce câmpuri de antet a adăugat serverul și unde apare plicul în fișierul stocat (`X-Peer`, `X-MailFrom`, `X-RcptTo`)?
- **Q3.** Cum apare în mesajul stocat linia pe care ați tastat-o cu două puncte? De ce a fost necesar al doilea punct? Începeți un mesaj nou cu o linie de corp `.` urmată de mai mult text: ce se întâmplă?

Sarcina 2 — MIME: un atașament pe fir

```
"""send_mail.py - trimite prin SMTP un mesaj MIME cu un atașament (Laboratorul 5).
utilizare: python3 send_mail.py [server] [port] [atașament]
       (server implicit 127.0.0.1, port 8025)"""
```

```
import mimetypes, smtplib, sys
from email.message import EmailMessage
from pathlib import Path

server = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
port = int(sys.argv[2]) if len(sys.argv) > 2 else 8025

msg = EmailMessage()                          # câmpurile de antet RFC 5322
msg["From"] = "Alice Student <alice@lab.test>"
msg["To"] = "Bob Teacher <bob@lab.test>"
msg["Subject"] = "Lab 5 report — first draft"     # non-ASCII: RFC 2047
msg.set_content("Hello Bob,\n\nthe report is attached.\n\nAlice\n")
if len(sys.argv) > 3:                 # MIME: corpul devine multipart
    path = Path(sys.argv[3])
    ctype = mimetypes.guess_type(path.name)[0] or "application/octet-stream"
    maintype, subtype = ctype.split("/", 1)
    msg.add_attachment(path.read_bytes(), maintype=maintype,
                       subtype=subtype, filename=path.name)

with smtplib.SMTP(server, port) as smtp:
    smtp.set_debuglevel(1)                    # afișează dialogul SMTP
    smtp.ehlo("client.lab.test")
    # plicul (MAIL FROM, RCPT TO) este separat de From:/To:
    smtp.send_message(msg, from_addr="alice@lab.test",
                      to_addrs=["bob@lab.test"])
```

```
head -c 30000 /dev/urandom > photo.bin          # 30 000 de octeți aleatori
python3 python/send_mail.py 127.0.0.1 8025 photo.bin
ls -l photo.bin ~/Maildir/new/
less "$(ls -t ~/Maildir/new/* | head -1)"       # cel mai nou mesaj
```

- **Q4.** În ieșirea de depanare, identificați plicul (`MAIL FROM`, `RCPT TO`) și începutul mesajului după DATA. Cum a fost scris în antet subiectul non-ASCII (linia de pauză —)? Decodificați-l manual sau cu `python3 -c "import base64; print(base64.b64decode('…'))"`.
- **Q5.** Găsiți `Content-Type` al întregului mesaj și al fiecărei părți, delimitatorul (boundary) și `Content-Transfer-Encoding` al atașamentului. Comparați dimensiunile: cu cât la sută este mai mare mesajul stocat decât `photo.bin`? Corespunde cu cei 37 % din curs?

Sarcina 3 — Citirea poștei: POP3 și IMAP

`pop3_server.py` servește mesajele din `~/Maildir/new` utilizatorului bob, cu parola `secret`. Implementează cele trei stări din curs și nu șterge nimic până la `QUIT`.

```
# terminalul C
python3 python/pop3_server.py ~/Maildir 1110
# terminalul B
python3 python/talk.py 127.0.0.1 1110
```

```
USER bob
PASS secret
STAT
LIST
UIDL
TOP 1 0
RETR 1
DELE 1
RSET
DELE 1
QUIT
```

Acum priviți un server IMAP real, peste TLS. Cereți doar capabilitățile sale și vă deconectați — nu este nevoie de niciun cont:

```
python3 python/talk.py imap.gmail.com 993 --tls
a1 CAPABILITY
a2 LOGOUT
openssl s_client -connect imap.gmail.com:993 -quiet     # același lucru, cu openssl
```

- **Q6.** Dați răspunsul POP3 la STAT și LIST și explicați numerele. Cum marchează RETR sfârșitul mesajului? Ce s-ar întâmpla cu mesajul 1 dacă ați închide conexiunea fără QUIT? Verificați cu `ls ~/Maildir/new` după o sesiune încheiată cu `DELE 1` și QUIT.
- **Q7.** Copiați mesajul de întâmpinare IMAP și linia CAPABILITY. Găsiți `IMAP4rev1`, `IDLE` și metodele de autentificare (AUTH=…). Ce linii sunt etichetate și care neetichetate? De ce nu puteți vedea acest dialog cu tcpdump?

Sarcina 4 — DNS cu dig

```
dig www.unibuc.ro                          # cele patru secțiuni; resolverul folosit
dig +short www.unibuc.ro A; dig +short unibuc.ro MX; dig +short unibuc.ro NS
dig +trace www.unibuc.ro                   # rezolvare iterativă de la rădăcină
dig +norecurse @a.root-servers.net www.unibuc.ro   # o trimitere
dig @1.1.1.1 www.google.com A              # de două ori, la câteva secunde distanță: TTL
dig gmail.com TXT; dig _dmarc.gmail.com TXT        # SPF și DMARC
dig -x 8.8.8.8                             # căutare inversă: PTR
dig +dnssec unibuc.ro SOA                  # înregistrări RRSIG, dacă zona este semnată
curl -s -H 'accept: application/dns-json' \
    'https://cloudflare-dns.com/dns-query?name=unibuc.ro&type=MX'
```

- **Q8.** În primul `dig`, citiți linia de antet: ce indicatori sunt setați (qr, rd, ra, aa)? Ce server a răspuns și cât a durat? Repetați comanda: de ce este timpul mai scurt acum?
- **Q9.** În ieșirea `+trace`, numiți serverele care au răspuns la fiecare nivel și tipurile de înregistrări care au trimis resolverul mai departe. În interogarea `+norecurse` către rădăcină, în ce secțiune se află serverele `.ro` și de ce există o secțiune Additional?
- **Q10.** Cum s-a schimbat TTL-ul lui www.google.com între cele două interogări către 1.1.1.1? Ce spun înregistrările SPF și DMARC ale lui gmail.com? De ce nu apare interogarea DoH în `tcpdump port 53`?

Sarcina 5 — Un client DNS în C

Programul construiește o interogare exact ca în secțiunea 8.4 a cursului — antetul de 12 octeți în ordinea octeților de rețea, numele sub formă de etichete, QTYPE și QCLASS —, o trimite într-o singură datagramă UDP, așteaptă răspunsul până la 2 secunde (trei încercări) și decodifică fiecare înregistrare de resursă, urmând pointerii de compresie în `read_name`.

```
/* dns_query.c - construiește manual o interogare DNS și o trimite peste UDP
   (Rețele de calculatoare, Laboratorul 5)
   utilizare: ./dns_query nume [tip] [server] [port]
   de ex.     ./dns_query www.kernel.org AAAA 1.1.1.1                        */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include <time.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/time.h>

static const struct { const char *name; int code; } types[] = {
    {"A", 1}, {"NS", 2}, {"CNAME", 5}, {"SOA", 6},
    {"PTR", 12}, {"MX", 15}, {"TXT", 16}, {"AAAA", 28} };
```

```
#define NTYPES (int)(sizeof types / sizeof types[0])

static const char *type_name(int code) {
    for (int i = 0; i < NTYPES; i++)
        if (types[i].code == code) return types[i].name;
    return "?";
}

/* Citește un număr pe 16 biți stocat în ordinea octeților de rețea. */
static uint16_t get16(const unsigned char *p) {
    return (uint16_t)(p[0] << 8 | p[1]);
}

/* Citește un nume (eventual comprimat) de la msg[*pos] în out și mută *pos
   după el. Un octet 11xxxxxx începe un pointer de 2 octeți către un nume anterior. */
static int read_name(const unsigned char *msg, int len, int *pos,
                     char *out, int cap) {
    int p = *pos, jumped = 0, n = 0, hops = 0;
    for (;;) {
        if (p >= len || hops++ > 50) return -1;
        int l = msg[p];
        if ((l & 0xC0) == 0xC0) {                /* pointer: offset pe 14 biți */
            if (p + 1 >= len) return -1;
            if (!jumped) *pos = p + 2;
            p = ((l & 0x3F) << 8) | msg[p + 1];
            jumped = 1;
            continue;
        }
        if (l == 0) {                            /* eticheta rădăcină: sfârșitul */
            if (!jumped) *pos = p + 1;
            break;
        }
        if (p + 1 + l > len || n + l + 2 > cap) return -1;
        memcpy(out + n, msg + p + 1, l);         /* copiază o etichetă */
        n += l;
        out[n++] = '.';
        p += 1 + l;
    }
    if (n == 0) strcpy(out, ".");                /* rădăcina însăși */
    else out[n - 1] = '\0';                      /* elimină punctul final */
    return 0;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "usage: %s name [type] [server] [port]\n", argv[0]);
        return 1;
    }
    const char *qname = argv[1], *server = argc > 3 ? argv[3] : "1.1.1.1";
    int qtype = 0, port = argc > 4 ? atoi(argv[4]) : 53;
    for (int i = 0; i < NTYPES; i++)
        if (strcasecmp(argc > 2 ? argv[2] : "A", types[i].name) == 0)
            qtype = types[i].code;
    if (!qtype) { fprintf(stderr, "unknown type\n"); return 1; }

    /* --- interogarea: un antet de 12 octeți, apoi întrebarea;
           fiecare număr în ordinea octeților de rețea --- */
    unsigned char q[512];
    int n = 0;
    srand((unsigned)time(NULL) ^ (unsigned)getpid());
    uint16_t id = (uint16_t)rand();
    uint16_t hdr[6] = { htons(id), htons(0x0100), /* indicatori: RD = 1 */
                        htons(1), 0, 0, 0 };     /* o întrebare */
    memcpy(q, hdr, 12);
    n = 12;
    /* numele sub formă de etichete: www.kernel.org -> 3 www 6 kernel 3 org 0 */
```

```
for (const char *s = qname; *s; ) {
    const char *dot = strchr(s, '.');
    size_t l = dot ? (size_t)(dot - s) : strlen(s);
    if (l == 0 || l > 63 || n + l + 6 > sizeof q) {
        fprintf(stderr, "bad name\n");
        return 1;
    }
    q[n++] = (unsigned char)l;
    memcpy(q + n, s, l);
    n += (int)l;
    s += l + (dot ? 1 : 0);
}
q[n++] = 0;
uint16_t tc[2] = { htons((uint16_t)qtype), htons(1) };  /* QCLASS IN */
memcpy(q + n, tc, 4);
n += 4;
printf("query (%d bytes):", n);
for (int i = 0; i < n; i++) printf(" %02x", q[i]);
printf("\n");

/* --- o datagramă UDP trimisă, una primită: fără conexiune, fără handshake --- */
int fd = socket(AF_INET, SOCK_DGRAM, 0);
struct sockaddr_in to = {0};
to.sin_family = AF_INET;
to.sin_port = htons((uint16_t)port);
if (inet_pton(AF_INET, server, &to.sin_addr) != 1) {
    fprintf(stderr, "the server must be an IPv4 address\n");
    return 1;
}
struct timeval tv = {2, 0};                   /* așteaptă cel mult 2 s */
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
unsigned char r[4096];
ssize_t len = -1;
for (int attempt = 1; attempt <= 3 && len < 0; attempt++) {
    sendto(fd, q, (size_t)n, 0, (struct sockaddr *)&to, sizeof to);
    len = recvfrom(fd, r, sizeof r, 0, NULL, NULL);
    if (len < 0) printf("no answer after 2 s (attempt %d)\n", attempt);
}                                             /* UDP o poate pierde: reîncercăm */
close(fd);
if (len < 12) return 1;

uint16_t f[6];                                /* antetul, în ordinea host-ului */
for (int i = 0; i < 6; i++) f[i] = get16(r + 2 * i);
if (f[0] != id) { fprintf(stderr, "wrong id\n"); return 1; }
printf("response (%zd bytes): id=0x%04x flags=0x%04x rcode=%d%s%s\n",
       len, f[0], f[1], f[1] & 0xF, (f[1] & 0x0400) ? " AA" : "",
       (f[1] & 0x0200) ? " TC" : "");
printf("question=%d answer=%d authority=%d additional=%d\n",
       f[2], f[3], f[4], f[5]);

int pos = 12;
char name[256];
for (int i = 0; i < f[2]; i++) {              /* sare peste întrebare (întrebări) */
    if (read_name(r, (int)len, &pos, name, sizeof name) < 0) return 1;
    pos += 4;
}
for (int i = 0; i < f[3] + f[4] + f[5]; i++) {   /* înregistrări de resurse */
    if (read_name(r, (int)len, &pos, name, sizeof name) < 0) return 1;
    if (pos + 10 > len) return 1;
    uint16_t t = get16(r + pos), rdlen = get16(r + pos + 8);
    uint32_t ttl = (uint32_t)get16(r + pos + 4) << 16 | get16(r + pos + 6);
    pos += 10;
    if (pos + rdlen > len) return 1;
    char val[256] = "", ex[200];
    int vp = pos;
    if (t == 1 && rdlen == 4)
```

```
            inet_ntop(AF_INET, r + pos, val, sizeof val);
        else if (t == 28 && rdlen == 16)
            inet_ntop(AF_INET6, r + pos, val, sizeof val);
        else if (t == 2 || t == 5 || t == 12)       /* NS, CNAME, PTR */
            read_name(r, (int)len, &vp, val, sizeof val);
        else if (t == 15) {                         /* MX: preferință, nume */
            vp += 2;
            read_name(r, (int)len, &vp, ex, sizeof ex);
            snprintf(val, sizeof val, "%d %s", get16(r + pos), ex);
        } else if (t == 16)                         /* TXT: primul șir */
            snprintf(val, sizeof val, "\"%.*s\"", r[pos],
                     (const char *)r + pos + 1);
        else
            snprintf(val, sizeof val, "(%d bytes of data)", rdlen);
        const char *sec = i < f[3] ? "answer"
                        : i < f[3] + f[4] ? "authority" : "additional";
        printf("%-10s %-26s %6u  %-5s %s\n", sec, name, ttl,
               type_name(t), val);
        pos += rdlen;
    }
    return 0;
}
```

```
make -C linux
# terminalul C: urmăriți portul 53
sudo tcpdump -i any -n -X udp port 53
# terminalul B
./linux/dns_query www.unibuc.ro A 1.1.1.1
./linux/dns_query unibuc.ro MX
./linux/dns_query gmail.com TXT 8.8.8.8
./linux/dns_query www.unibuc.ro A 198.41.0.4   # a.root-servers.net: trimitere
python3 python/dns_query.py www.unibuc.ro AAAA            # același lucru în Python
```

- **Q11.** În ieșirea tcpdump a primei interogări, marcați ID-ul, indicatorii (0x0100: ce este setat?), numărătorile și octeții întrebării. Comparați cu interogarea din tema dumneavoastră pentru unibuc.ro MX.
- **Q12.** În răspuns, găsiți un pointer de compresie (`c0 ..`) și spuneți către ce offset indică și ce nume se află acolo. Cum deosebește `read_name` un pointer de lungimea unei etichete și cum evită o buclă infinită în cazul unui răspuns rău intenționat?
- **Q13.** Interogați direct `a.root-servers.net` (198.41.0.4; programul acceptă doar adrese) — acolo RD este ignorat. De ce este ANCOUNT 0 și ce găsiți în secțiunile Authority și Additional? Extindeți programul: când răspunsul este o trimitere, interogați primul server din secțiunea Additional și tot așa până sosește un răspuns — propriul resolver iterativ. (Indiciu: puneți bucla în jurul lui `sendto`/`recvfrom`; păstrați înregistrările A din secțiunea Additional.)

Sarcina 6 — Un router NAT propriu

`nat_lab.sh` construiește trei spații de nume de rețea — stive de rețea izolate în interiorul mașinii dumneavoastră — conectate prin cabluri Ethernet virtuale: o rețea de acasă `lan`, un `router` cu o interfață privată și una publică și un `server` în Internet. Routerul redirecționează pachetele și ascunde (masquerade) LAN-ul în spatele adresei sale publice cu o singură regulă nftables. Serverul nu are rută către rețeaua privată, ca în Internetul real.

```
#!/bin/sh
# nat_lab.sh - o rețea de acasă, un router NAT și un server în interiorul unei singure
# mașini Linux, construite din spații de nume de rețea (Laboratorul 5).
# utilizare: sudo ./nat_lab.sh up | down
#
#  [lan] 192.168.77.2 --- 192.168.77.1 [router]               (partea privată)
#                         203.0.113.1  [router] --- 203.0.113.10 [server]
```

```
#                          routerul face masquerade: NAT      (partea publică)
set -e
if [ "$1" = "down" ]; then
    for ns in lan router server; do ip netns del $ns 2>/dev/null || true; done
    echo "removed"; exit 0
fi
[ "$1" = "up" ] || { echo "usage: $0 up | down"; exit 1; }

for ns in lan router server; do
    ip netns add $ns; ip -n $ns link set lo up
done
# două cabluri virtuale: lan <-> router și router <-> server
ip link add veth-lan netns lan type veth peer name veth-in netns router
ip link add veth-out netns router type veth peer name veth-srv netns server

ip -n lan    addr add 192.168.77.2/24 dev veth-lan
ip -n router addr add 192.168.77.1/24 dev veth-in
ip -n router addr add 203.0.113.1/24  dev veth-out
ip -n server addr add 203.0.113.10/24 dev veth-srv
ip -n lan    link set veth-lan up
ip -n router link set veth-in up
ip -n router link set veth-out up
ip -n server link set veth-srv up

ip -n lan route add default via 192.168.77.1     # gateway-ul implicit al lan
ip netns exec router sysctl -q -w net.ipv4.ip_forward=1
# Serverul nu are rută înapoi către 192.168.77.0/24: fără NAT,
# răspunsurile sale nu s-ar putea întoarce.

# NAT: pachetele care pleacă spre partea publică primesc adresa routerului
ip netns exec router nft -f - <<'NFT'
table ip nat {
    chain postrouting {
        type nat hook postrouting priority srcnat; policy accept;
        oifname "veth-out" ip saddr 192.168.77.0/24 masquerade
    }
}
NFT
echo "ready: lan 192.168.77.2 -> router (NAT) -> server 203.0.113.10"
```

```
chmod +x linux/nat_lab.sh; sudo ./linux/nat_lab.sh up
# terminalul C: un server web în „Internet”
sudo ip netns exec server python3 -m http.server 80 --bind 203.0.113.10
# terminalul B: un client în rețeaua de acasă
sudo ip netns exec lan curl -sI http://203.0.113.10/ | head -1
sudo ip netns exec router conntrack -L          # tabela de translatare
sudo ip netns exec router nft list ruleset
```

Eliminați NAT și încercați din nou, apoi reconstruiți laboratorul și adăugați **redirecționarea porturilor**: un server web din interiorul rețelei de acasă devine accesibil din exterior pe portul 8080 al routerului.

```
sudo ip netns exec router nft flush table ip nat
sudo ip netns exec lan curl -m 3 http://203.0.113.10/     # expiră: de ce?
sudo ./linux/nat_lab.sh down; sudo ./linux/nat_lab.sh up
sudo ip netns exec lan python3 -m http.server 8080        # terminalul C
sudo ip netns exec router nft add chain ip nat prerouting \
    '{ type nat hook prerouting priority dstnat; policy accept; }'
sudo ip netns exec router nft add rule ip nat prerouting \
    iifname veth-out tcp dport 8080 dnat to 192.168.77.2:8080
sudo ip netns exec server curl -s http://203.0.113.1:8080/ | head -3
sudo ./linux/nat_lab.sh down                              # la final
```

În final, găsiți maparea pe care o creează NAT-urile rețelei dumneavoastră reale. Un server STUN raportează adresa și portul de la care a sosit pachetul dumneavoastră UDP:

```
"""stun_client.py - întreabă un server STUN de la ce adresă și port publice
provin pachetele dumneavoastră UDP, adică ce mapare a creat NAT-ul (Laboratorul 5).
utilizare: python3 stun_client.py [server] [port]
       (implicit stun.l.google.com 19302; cerere Binding RFC 8489)"""
import os, socket, struct, sys

server = sys.argv[1] if len(sys.argv) > 1 else "stun.l.google.com"
port = int(sys.argv[2]) if len(sys.argv) > 2 else 19302
MAGIC = 0x2112A442

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.settimeout(3)
sock.bind(("0.0.0.0", 0))
print("local (private) side:", sock.getsockname())
for attempt in range(2):                  # două cereri de pe același socket
    tid = os.urandom(12)                  # id-ul tranzacției
    request = struct.pack("!HHI", 0x0001, 0, MAGIC) + tid   # cerere Binding
    sock.sendto(request, (server, port))
    data, src = sock.recvfrom(2048)
    mtype, length, magic = struct.unpack("!HHI", data[:8])
    assert data[8:20] == tid and mtype == 0x0101, "not our Binding response"
    pos = 20
    while pos < 20 + length:              # atribute: tip, lungime, valoare
        atype, alen = struct.unpack("!HH", data[pos:pos + 4])
        value = data[pos + 4:pos + 4 + alen]
        if atype == 0x0020:               # XOR-MAPPED-ADDRESS
            xport = struct.unpack("!H", value[2:4])[0] ^ (MAGIC >> 16)
            xaddr = struct.unpack("!I", value[4:8])[0] ^ MAGIC
            print(f"request {attempt + 1}: the server saw us as "
                  f"{socket.inet_ntoa(struct.pack('!I', xaddr))}:{xport}")
        pos += 4 + (alen + 3) // 4 * 4    # valorile sunt completate (padding) la 4
octeți

ip -4 addr show | grep inet                      # partea dumneavoastră privată
python3 python/stun_client.py                    # partea dumneavoastră publică
```

- **Q14.** Ce adresă de client a înregistrat serverul web: 192.168.77.2 sau 203.0.113.1? Copiați intrarea conntrack și explicați cele două jumătăți ale ei (direcția originală și direcția răspunsului). De ce a expirat cererea după `flush`?
- **Q15.** Cu redirecționarea porturilor, ce adresă de client a înregistrat serverul web din `lan`? Ce lanț (chain) și ce hook nftables a folosit fiecare regulă și de ce trebuie ca DNAT să aibă loc *înainte* de rutare, iar masquerading-ul *după* aceasta?
- **Q16.** Comparați adresa dumneavoastră privată cu adresa raportată de STUN. Se află mașina dumneavoastră în spatele NAT? În spatele CGNAT (este adresa WAN a routerului în 100.64.0.0/10)? Au primit ambele cereri același port public — ce spune acest lucru despre NAT (mapare independentă de punctul final)?

Dacă ceva nu funcționează

Simptom	Cauza probabilă și ce să încercați
`No module named aiosmtpd`	instalați `python3-aiosmtpd` (Ubuntu universe) sau `pip install --user aiosmtpd`
talk.py se încheie înainte de ultimul răspuns	așteaptă 5 s după ultima linie; apăsați Ctrl-C când ați terminat sau folosiți `nc -C`
`dig: command not found`	instalați `dnsutils` (Debian/Ubuntu) sau `bind-utils` (Fedora)
`dig +trace` sau Sarcina 5 expiră	rețeaua blochează DNS-ul de ieșire către alte servere decât propriul resolver: folosiți ca argument server resolverul din `/etc/resolv.conf` (sau din `resolvectl status`)

Simptom	Cauza probabilă și ce să încercați
tcpdump arată doar interogări către 127.0.0.53	systemd-resolved este resolverul stub local; capturați cu `-i any` și interogați explicit un server extern
`nat_lab.sh`: File exists	spațiile de nume au rămas de la o rulare anterioară: rulați mai întâi `sudo ./linux/nat_lab.sh down`
`conntrack: command not found` sau tabelă goală	instalați `conntrack`; listați tabela imediat după curl, înainte să expire intrarea
STUN expiră	UDP-ul de ieșire este blocat în această rețea (unele firewall-uri din campusuri și companii): încercați de acasă sau de pe un hotspot mobil

PARTEA D · SĂPTĂMÂNA 5

Laboratorul 5 (Windows): aceleași protocoale cu instrumente Windows și Winsock; găsirea propriului NAT

Fișă de laborator de două ore (2 × 50 de minute) care însoțește Cursul 5

Această fișă urmează fișa Linux sarcină cu sarcină, astfel încât răspunsurile să poată fi comparate. Serverele de poștă sunt aceleași programe Python; DNS este explorat cu `Resolve-DnsName` din PowerShell, cu `nslookup` și cu cache-ul DNS propriu al Windows; clientul DNS este versiunea Winsock a programului în C. Spațiile de nume de rețea nu există pe Windows, așa că, în loc să construiți un router NAT, găsiți NAT-urile dintre PC-ul dumneavoastră și Internet — și pe cel pe care îl rulează Windows însuși pentru WSL.

La sfârșitul laboratorului ar trebui să puteți

- trimite un mesaj tastând SMTP, precum și un mesaj MIME cu un atașament;
- citi și șterge manual poșta cu POP3 și deschide o sesiune IMAP peste TLS;
- interoga DNS cu Resolve-DnsName și nslookup și inspecta și goli cache-ul DNS al Windows;
- construi și extinde un client DNS cu Winsock și compara pachetele lui în Wireshark;
- afla câte NAT-uri separă PC-ul dumneavoastră de Internet și ce mapare publică creează acestea.

Înainte de a începe

- **Python 3** de pe python.org (bifați „Add to PATH”); apoi `py -m pip install aiosmtpd`.
- **Un compilator C**, ca în Laboratoarele 3 și 4: MinGW-w64 (`gcc ... -lws2_32`) sau Visual Studio Build Tools (`cl ... ws2_32.lib`).
- **Wireshark** cu Npcap (cu suport pentru capturarea traficului loopback).
- **Fișiere.** Dezarhivați arhiva în `lab5\`; folderul Windows conține `dns_query_win.c`; programele Python se află în `python\`. Lucrați în PowerShell din `lab5\`.
- **Ce predați.** Un scurt raport cu răspunsurile la **Q1–Q16** și ieșirile care le susțin; fișierul `dns_query_win.c` extins de dumneavoastră.

Plan

Pas	Activitate	Timp
1	SMTP de mână: un server de poștă propriu	15 min
2	MIME: un atașament pe fir	10 min
3	Citirea poștei: POP3 de mână, mesajul de întâmpinare IMAP	15 min
	Pauză	10 min
4	DNS cu Resolve-DnsName, nslookup și cache-ul Windows	15 min
5	Clientul DNS cu Winsock	20 min
6	Găsirea propriului NAT; STUN	15 min

Sarcina 1 — SMTP de mână

```
# fereastra A: un server SMTP pe portul 8025, care stochează poșta în .\Maildir
py -m aiosmtpd -n -l 127.0.0.1:8025 -c aiosmtpd.handlers.Mailbox Maildir
# fereastra B: vorbiți SMTP (tastați liniile din fișa Linux, Sarcina 1)
py python\talk.py 127.0.0.1 8025
Get-ChildItem Maildir\new; Get-Content (Get-ChildItem Maildir\new)[0].FullName
```

Tastați EHLO `client.lab.test`, MAIL FROM:<alice@lab.test>, RCPT TO:<bob@lab.test>, RCPT TO:<carol@lab.test>, DATA, trei linii de antet, o linie goală, un corp care conține o linie ce începe cu două puncte, o linie cu un singur punct și QUIT — ca în fișa Linux. Dacă este instalat clientul TELNET al Windows, funcționează și `telnet 127.0.0.1 8025`, dar acesta trimite fiecare tastă imediat ce o apăsați.

- **Q1.** Copiați răspunsul serverului la EHLO. Ce coduri de răspuns ați primit pentru MAIL, RCPT, DATA, punctul final și QUIT și din ce clasă face parte fiecare?
- **Q2.** Carol nu este numită în nicio linie de antet. Primește ea mesajul? Unde apare plicul în fișierul stocat?
- **Q3.** Cum apare în mesajul stocat linia pe care ați tastat-o cu două puncte și de ce?

Sarcina 2 — MIME: un atașament pe fir

`send_mail.py` este listat în fișa Linux; construiește mesajul cu pachetul `email` din Python și afișează dialogul SMTP.

```
$b = New-Object byte[] 30000; (New-Object Random).NextBytes($b)
[IO.File]::WriteAllBytes("$PWD\photo.bin", $b)
py python\send_mail.py 127.0.0.1 8025 photo.bin
Get-ChildItem photo.bin, Maildir\new | Format-Table Name, Length
notepad (Get-ChildItem Maildir\new | Sort-Object LastWriteTime)[-1].FullName
```

- **Q4.** Identificați plicul și începutul mesajului în ieșirea de depanare. Cum a fost scrisă linia de pauză din subiect? Decodificați-o: `[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String('4oCU'))`.
- **Q5.** Găsiți `Content-Type` al mesajului și al fiecărei părți, delimitatorul (boundary) și codificarea de transfer a atașamentului. Cu cât la sută este mai mare mesajul stocat decât `photo.bin`?

Sarcina 3 — Citirea poștei: POP3 și IMAP

```
py python\pop3_server.py Maildir 1110          # fereastra C
py python\talk.py 127.0.0.1 1110               # fereastra B
py python\talk.py imap.gmail.com 993 --tls     # apoi: a1 CAPABILITY, a2 LOGOUT
```

În sesiunea POP3 tastați USER bob, PASS `secret`, STAT, LIST, UIDL, TOP 1 0, RETR 1, DELE 1, RSET, DELE 1, QUIT. Capturați sesiunea în Wireshark pe adaptorul loopback cu filtrul `tcp.port == 1110`, iar sesiunea IMAP pe adaptorul de rețea cu `tcp.port == 993`.

- **Q6.** Explicați răspunsurile la STAT și LIST. Cum marchează RETR sfârșitul mesajului? Este parola POP3 lizibilă în captură? Ce mesaje rămân în `Maildir\new` după QUIT?
- **Q7.** Copiați mesajul de întâmpinare IMAP și linia CAPABILITY; găsiți `IMAP4rev1`, `IDLE` și metodele AUTH=. Ce arată Wireshark din sesiunea IMAP? Ce nume de server mai puteți citi și în ce mesaj TLS?

Sarcina 4 — DNS cu instrumentele Windows

```
Resolve-DnsName www.unibuc.ro                       # implicit tipul A
Resolve-DnsName unibuc.ro -Type MX
Resolve-DnsName unibuc.ro -Type NS
Resolve-DnsName gmail.com -Type TXT -Server 1.1.1.1
Resolve-DnsName _dmarc.gmail.com -Type TXT
Resolve-DnsName 8.8.8.8                             # invers: PTR
Resolve-DnsName www.unibuc.ro -Server a.root-servers.net -NoRecursion
nslookup -type=soa unibuc.ro
nslookup -debug www.google.com 1.1.1.1              # antetul și TTL-urile
ipconfig /displaydns | Select-String -Context 0,6 'unibuc'
ipconfig /flushdns                                  # (admin) golește cache-ul
Get-DnsClientServerAddress -AddressFamily IPv4      # resolverele dumneavoastră
```

- **Q8.** Ce resolver folosește PC-ul dumneavoastră? Rulați `nslookup -debug www.google.com 1.1.1.1` de două ori, la câteva secunde distanță: cum s-a schimbat TTL-ul și de ce?
- **Q9.** Interogați `a.root-servers.net` cu `-NoRecursion`. Ce servere numește răspunsul, în ce secțiune, și de ce conține și adrese? Continuați manual până la răspunsul autoritativ, ca în tema 2 a seminarului.
- **Q10.** Găsiți www.unibuc.ro în `ipconfig /displaydns`: ce TTL arată cache-ul Windows? Care sunt înregistrările SPF și DMARC ale lui gmail.com și ce îi cer ele unui server destinatar?

Sarcina 5 — Clientul DNS cu Winsock

`dns_query_win.c` este programul Linux din Laboratorul 5 cu modificările Winsock pe care le cunoașteți: `WSAStartup`, `SOCKET`, `closesocket` și un timp de așteptare la recepție dat ca `DWORD` în milisecunde în loc de un `struct timeval`. Citiți listingul complet în fișa Linux; liniile care diferă sunt:

```
#include <winsock2.h>
#include <ws2tcpip.h>
...
WSADATA wsa;
if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) return 1;
SOCKET fd = socket(AF_INET, SOCK_DGRAM, 0);
DWORD tv = 2000;                /* Winsock: milisecunde, nu timeval */
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, (const char *)&tv, sizeof tv);
...
closesocket(fd);
WSACleanup();
```

```
gcc -Wall -O2 windows\dns_query_win.c -o dns_query.exe -lws2_32
# sau: cl windows\dns_query_win.c ws2_32.lib
.\dns_query.exe www.unibuc.ro A 1.1.1.1
.\dns_query.exe unibuc.ro MX
.\dns_query.exe www.unibuc.ro A 198.41.0.4      # a.root-servers.net
py python\dns_query.py www.unibuc.ro AAAA
```

Capturați pe adaptorul de rețea cu filtrul de afișare Wireshark `dns`. Wireshark decodifică fiecare câmp; comparați arborele său cu ieșirea programului dumneavoastră și cu octeții din panoul de jos.

- **Q11.** În Wireshark, marcați ID-ul, indicatorii, numărătorile și octeții întrebării din interogarea dumneavoastră. Este răspunsul asociat interogării? După ce câmp?
- **Q12.** Găsiți un pointer de compresie în răspuns (`c0 ..`): către ce offset indică și ce nume se află acolo? Cum deosebește `read_name` un pointer de lungimea unei etichete?
- **Q13.** De ce este ANCOUNT 0 în răspunsul lui `a.root-servers.net`? Extindeți programul într-un resolver iterativ care urmează trimiterile, așa cum este descris în Q13 din fișa Linux.

Sarcina 6 — Găsirea propriului NAT

```
ipconfig | Select-String 'IPv4|Gateway'            # partea dumneavoastră privată
curl.exe -s https://ifconfig.me; ''                # adresa pe care o văd serverele
tracert -d -h 4 1.1.1.1                            # primele routere
py python\stun_client.py                          # maparea UDP
py python\stun_client.py stun.cloudflare.com 3478
```

Deschideți pagina de stare a routerului de acasă (de obicei la adresa gateway-ului) și găsiți adresa sa WAN (Internet). Comparați cele trei adrese: a PC-ului dumneavoastră, adresa WAN a routerului și cea raportată de `ifconfig.me` și de STUN. Dacă adresa WAN a routerului este privată sau în **100.64.0.0/10**, ISP-ul dumneavoastră folosește NAT la nivel de operator și există două NAT-uri pe traseu.

Windows rulează un NAT propriu: **WSL 2** și *Default Switch* din Hyper-V îi dau mașinii virtuale Linux o adresă privată în spatele unui NAT din interiorul Windows. Dacă aveți WSL 2, comparați `ip -4 addr` din WSL cu `ipconfig` din Windows (adaptorul *vEthernet (WSL)*) și rulați `curl ifconfig.me` în ambele.

- **Q14.** Scrieți adresa PC-ului dumneavoastră, gateway-ul, adresa WAN a routerului și adresa publică raportată de ifconfig.me. Care sunt private (RFC 1918 sau 100.64.0.0/10)? Câte NAT-uri sunt pe traseul dumneavoastră?
- **Q15.** Raportează STUN aceeași adresă ca ifconfig.me? Au văzut ambele cereri STUN de pe același socket același port public? A fost portul același cu cel local? Ce spune acest lucru despre NAT-ul dumneavoastră?
- **Q16.** (WSL 2) Desenați traseul unui pachet de la WSL la ifconfig.me, cu fiecare adresă și fiecare translatare. Ar putea fi accesat un program din WSL de pe un alt PC din LAN fără configurare? (Windows numește soluția `netsh interface portproxy` — o regulă de redirecționare a porturilor.)

Dacă ceva nu funcționează

Simptom	Cauza probabilă și ce să încercați
`'py' is not recognized`	Python nu este instalat sau nu se află în PATH; reinstalați de pe python.org cu „Add to PATH” sau folosiți `python`
aiosmtpd: `address already in use`	alt server folosește portul 8025: alegeți alt port în ambele ferestre
talk.py se încheie înainte de ultimul răspuns	așteaptă 5 s după ultima linie; apăsați Ctrl-C când ați terminat
`Resolve-DnsName` către un server extern expiră	rețeaua permite DNS doar către propriul resolver: folosiți adresa din `Get-DnsClientServerAddress`
Wireshark nu arată pachete DNS	răspunsul a venit din cache-ul Windows (`ipconfig /flushdns`) sau browserul folosește DNS peste HTTPS
`curl` afișează un obiect în loc de text	ați tastat aliasul PowerShell: tastați întotdeauna `curl.exe`
STUN expiră	UDP-ul de ieșire este blocat în această rețea; încercați de acasă sau de pe un hotspot mobil

Săptămâna viitoare

Cu Cursul 5, nivelul aplicație este complet: fiecare subiect din lista programei — FTP, HTTP, HTTPS, SSH, SMTP, POP, IMAP, TELNET, Usenet, DNS și NAT — a fost acoperit în Săptămânile 3–5. Cursul 6

coboară la **nivelul transport**: cum multiplexează numerele de port multe aplicații peste o singură adresă, UDP și principiile transferului fiabil de date pe care este construit TCP. În Laboratorul 6 scrieți propriul protocol de transfer fiabil peste UDP și îl urmăriți cum se recuperează după pierderea pachetelor.